

MANUAL DE EXPERIMENTOS
LINGUAGEM PYTHON

PARTE I

Ramon Mayor Martins, Ph.D.
ramon.mayor at ifsc.edu.br
IFSC · Câmpus São José

@rmayormartins · [rmayormartins.github.io](https://github.com/rmayormartins)

Material de apoio às aulas de programação
Versão 2026/2

I. PRIMEIROS PASSOS

1	Estrutura fundamental	5
	Como executar os exemplos	
2	Entrada e saída de dados	9
	Três estilos de formatação	
3	Tipos de dados e variáveis	10
	Operações numéricas · Booleanos e valores verdadeiros	

II. CONTROLE DE FLUXO

4	Estruturas de decisão	11
5	Estruturas de repetição	14

III. COLEÇÕES E TEXTO

6	Estruturas de dados	21
	Listas · Tuplas · Dicionários · Conjuntos · Pilha e fila	
7	Compreensões	8
8	Strings	15
	Métodos de transformação · Formatação e alinhamento · Aplicações	

IV. ORGANIZAÇÃO DO CÓDIGO

9	Funções	18
	Parâmetros · Escopo · Funções como valores	
10	Módulos e pacotes	7
11	Programação orientada a objetos	9

V. DADOS, ROBUSTEZ E PRÁTICA

12	Manipulação de arquivos	9
	Caminhos com pathlib · Formatos de intercâmbio	
13	Tratamento de exceções	8
	Lendo um traceback	
14	Biblioteca padrão essencial	8
15	Estilo, depuração e testes	5
	PEP 8 em uma página · Depuração	
16	Projetos integradores	4

VI. APÊNDICES

A	Referência de operadores	0
B	Métodos essenciais por tipo	0
	Listas · Dicionários · Conjuntos · Funções internas mais usadas	
C	Formatação de saída	0
D	Cartão de consulta rápida	1
	Para continuar estudando	

[0] COMO USAR ESTE MANUAL

Um roteiro de laboratório, não um livro de teoria: cada assunto aparece como um experimento curto, pronto para ser digitado, executado e modificado.

Os códigos seguem a numeração **CÓDIGO seção.item**, de modo que qualquer exemplo pode ser citado em aula ou em uma lista de exercícios. Cada bloco preto traz o programa completo e, logo abaixo, a saída real produzida pelo interpretador: nenhuma saída deste manual foi digitada à mão, todas vieram da execução do próprio código em Python 3.12.

Os exemplos foram mantidos curtos de propósito. Quando um assunto exige um programa maior, ele aparece separado, no fim da seção ou na seção 16, para que o conceito nunca fique escondido dentro de um exercício longo demais.

Elementos da página

- * **Bloco preto:** o programa. Use o botão copiar no canto direito para levá-lo ao editor.
- * **Saída:** o que aparece no terminal ao executar aquele programa.
- * **Nota:** informação complementar que vale conhecer.
- * **Atenção:** armadilha frequente, do tipo que custa uma tarde de depuração.
- * **Dica:** atalho ou boa prática da comunidade Python.
- * **Sintaxe:** a forma geral da construção que está sendo apresentada.
- * **Experimentos propostos:** ao fim de cada seção, quatro tarefas curtas para fixar o conteúdo.

Roteiro sugerido

As seções 1 a 5 formam a base indispensável e devem ser feitas na ordem. As seções 6 a 8 tratam de coleções e texto, onde a maior parte dos exercícios de disciplina se resolve. As seções 9 a 11 organizam o código em funções, módulos e objetos. As seções 12 a 15 preparam o programa para o mundo real, com arquivos, erros e estilo. A seção 16 reúne projetos completos e os apêndices servem de consulta rápida durante as avaliações.

[*] MANUAL EM TRÊS PARTES

Esta é a **Parte I**, dedicada à linguagem em si, seções 1 a 16. A **Parte II** continua na seção 17 e trata dos pacotes de cálculo, gráfico e estatística: `math`, `random`, `statistics`, `numpy`, `matplotlib` e `pandas`. A **Parte III** vai da seção 25 à 32 e cobre aprendizado de máquina e profundo com `scikit-learn`, `keras` e `pytorch`.

[i] REQUISITOS

Python 3.10 ou superior. Alguns exemplos usam recursos recentes, como `match/case` (3.10) e anotações do tipo `list[float]` (3.9). Nenhum exemplo depende de biblioteca externa: tudo roda com a instalação padrão.

=====

|| BLOCO I

PRIMEIROS PASSOS

Escrever, executar, entrar e sair com dados, nomear valores.

=====

[1] ESTRUTURA FUNDAMENTAL

Como um programa Python é escrito, comentado e executado. As convenções desta seção valem para todo o restante do manual.

Python é uma linguagem **interpretada**, de **tipagem dinâmica** e **indentação significativa**. Não existe declaração de tipo, não existem chaves delimitando blocos e não há ponto e vírgula obrigatório no fim das linhas. O que organiza o programa é o recuo do texto.

Como executar os exemplos

- * **Terminal:** salve o arquivo como `exemplo.py` e rode `python3 exemplo.py`.
- * **Modo interativo:** digite `python3` e teste expressões linha a linha (ótimo para experimentar).
- * **Editor:** VS Code com a extensão Python, ou Thonny para quem está começando.
- * **Navegador:** Google Colab, sem instalar nada.

[i] VERSÃO USADA NESTE MANUAL

Os exemplos foram testados em **Python 3.12**. Recursos que exigem versão mínima específica estão sinalizados no próprio exemplo, como `match/case` (3.10) e `dataclasses` (3.7).

+-- #CÓDIGO 1.1 Primeiro programa -----

A função `print()` escreve na saída padrão e pula uma linha ao final.

```
# Uma linha ja e um programa completo.
print("Olá, mundo!")
print("Manual de Experimentos com Python")
```

--- SAÍDA ---

```
Olá, mundo!
Manual de Experimentos com Python
```

+-- #CÓDIGO 1.2 Comentários e docstrings -----

Aspas triplas na primeira linha de um módulo, função ou classe formam a *docstring*, acessível em tempo de execução por `__doc__` e por `help()`.

```
# Comentario de linha: tudo apos o # e ignorado.
print("Executando") # comentario no fim da linha

def area_circulo(raio):
    """Retorna a área de um círculo de raio informado."""
    return 3.14159 * raio ** 2

print(area_circulo(2))
print(area_circulo.__doc__)
```

--- SAÍDA ---

```
Executando
12.56636
Retorna a área de um círculo de raio informado.
```

+-- #CÓDIGO 1.3 A indentação define o bloco -----

Quatro espaços por nível é o padrão da comunidade (PEP 8). Nunca misture espaços e tabulações no mesmo arquivo.

```
temperatura = 31

if temperatura > 30:
    print("Dentro do bloco: está calor")
    print("Ainda dentro do bloco")

print("Fora do bloco: executa sempre")
```

```
--- SAÍDA ---
Dentro do bloco: está calor
Ainda dentro do bloco
Fora do bloco: executa sempre
```

+-- #CÓDIGO 1.4 Erro de indentação (proposital) -----

Vale conhecer a mensagem antes de encontrá-la em uma avaliação.

```
if 10 > 5:
    print("faltou recuar esta linha")
```

```
--- SAÍDA ---
File "exemplo.py", line 2
    print("faltou recuar esta linha")
    ^
IndentationError: expected an indented block after 'if' statement on line 1
```

+-- #CÓDIGO 1.5 Linhas longas e continuação -----

```
# Ponto e virgula no fim da linha nao e necessario.
nome = "Python"

# Forma preferida: quebrar dentro de parenteses.
total = (10 + 20 + 30 +
         40 + 50)

# Alternativa com barra invertida (use com moderacao).
media = 10 + 20 \
        + 30

print(nome, total, media)
```

```
--- SAÍDA ---
Python 150 60
```

[*] EXPLORE A LINGUAGEM POR DENTRO

`dir(objeto)` lista o que um objeto sabe fazer e `help(objeto)` mostra a documentação. No modo interativo, `help(str.upper)` resolve a maioria das dúvidas mais rápido do que uma busca na internet.

>> EXPERIMENTOS PROPOSTOS

- Escreva um programa que imprima seu nome, seu curso e o semestre em três linhas.
- Crie uma função `area_retangulo(base, altura)` com docstring e imprima `__doc__`.
- Remova propositalmente a indentação de um bloco e anote a mensagem de erro completa.

[2] ENTRADA E SAÍDA DE DADOS

Mostrar resultados na tela, ler dados do teclado e converter texto em número. Três estilos de formatação convivem em Python: aqui você vê os três e aprende qual usar.

+-- #CÓDIGO 2.1 print() com vários argumentos -----

Os argumentos são separados por um espaço automático.

```
print("Olá", "mundo", 2026)
print(42)
print("Soma:", 2 + 3)
```

--- SAÍDA ---

```
Olá mundo 2026
42
Soma: 5
```

+-- #CÓDIGO 2.2 print() com sep e end -----

sep troca o separador entre argumentos e end troca o que vem depois do último.

```
print("A", "B", "C", sep="-")
print("2026", "07", "22", sep="/")
```

```
print("Esta linha", end=" ")
print("continua aqui.")
```

```
print("Sem quebra no fim", end="")
print(" | e o resto vem junto")
```

--- SAÍDA ---

```
A-B-C
2026/07/22
Esta linha continua aqui.
Sem quebra no fim | e o resto vem junto
```

+-- #CÓDIGO 2.3 input() lê sempre uma string -----

Mesmo quando o usuário digita números, o retorno é texto. Repare no tipo exibido.

```
nome = input("Digite seu nome: ")
idade = input("Digite sua idade: ")
```

```
print("Olá,", nome)
print("Tipo do que foi digitado:", type(idade))
```

--- SAÍDA ---

```
Digite seu nome: Ramon
Digite sua idade: 38
Olá, Ramon
Tipo do que foi digitado: <class 'str'>
```

+-- #CÓDIGO 2.4 Convertendo a entrada -----

`int()` e `float()` convertem texto em número. Se o texto não representar um número, ocorre `ValueError` (tratado na seção 13).

```
idade = int(input("Idade: "))
altura = float(input("Altura em metros: "))

print("No ano que vem você terá", idade + 1)
print("Altura em centímetros:", altura * 100)
```

```
--- SAÍDA ---

Idade: 38
Altura em metros: 1.78
No ano que vem você terá 39
Altura em centímetros: 178.0
```

Três estilos de formatação

+-- #CÓDIGO 2.5 f-strings (recomendado) -----

Prefixo `f` antes das aspas. Qualquer expressão Python pode ir entre chaves.

```
nome = "Maria"
idade = 25
preco = 29.9

print(f"Nome: {nome}")
print(f"{nome} tem {idade} anos e no ano que vem terá {idade + 1}")
print(f"Preço: R$ {preco:.2f}")
```

```
--- SAÍDA ---

Nome: Maria
Maria tem 25 anos e no ano que vem terá 26
Preço: R$ 29.90
```

+-- #CÓDIGO 2.6 Controle fino dentro da f-string -----

A sintaxe é `{valor:formato}`. O sufixo `=` imprime a expressão e o resultado, útil durante a depuração.

```
valor = 1234.5678
taxa = 0.0725
item = "Cabo"

print(f"Moeda:      {valor:,.2f}")
print(f"Percentual: {taxa:.2%}")
print(f"Científico: {valor:.3e}")
print(f"Zeros:      {42:05d}")
print(f"Alinhado:    |{item:<10}|{item:>10}|{item:^10}|")
print(f"Depuração:   {valor=}")
```

```
--- SAÍDA ---

Moeda:      1,234.57
Percentual: 7.25%
 Científico: 1.235e+03
Zeros:      00042
Alinhado:    |Cabo      |      Cabo|   Cabo   |
Depuração:   valor=1234.5678
```

+-- #CÓDIGO 2.7 Método .format() -----

Estilo anterior às f-strings. Você ainda encontra muito código assim, e ele é útil quando o texto do modelo vem de fora do programa.

```
nome, idade = "João", 30

print("Nome: {}, Idade: {}".format(nome, idade))
print("Trocando a ordem: {1} anos, {0}".format(nome, idade))
print("Com nomes: {n} ganha R$ {:.2f}".format(n=nome, s=3500.5))
```

--- SAÍDA ---

```
Nome: João, Idade: 30
Trocando a ordem: 30 anos, João
Com nomes: João ganha R$ 3500.50
```

+-- #CÓDIGO 2.8 Operador % (estilo clássico) -----

Herança da linguagem C. Aparece em códigos antigos e em mensagens de log.

```
nome, idade, nota = "Ana", 28, 8.75

print("Nome: %s" % nome)
print("%s tem %d anos e nota %.1f" % (nome, idade, nota))
print("Largura fixa: |%10s|%3d|" % (nome, idade))
```

--- SAÍDA ---

```
Nome: Ana
Ana tem 28 anos e nota 8.8
Largura fixa: |        Ana| 28|
```

ESTILO	EXEMPLO	QUANDO USAR
f-string	f"{n}"	Padrão em código novo (Python 3.6+)
.format()	"{}".format(n)	Modelos de texto vindos de arquivo ou banco
%	"%s" % n	Código legado e módulo <code>logging</code>

+-- Tabela 2.1: os três estilos de formatação.

+-- #CÓDIGO 2.9 Programa integrador: média de duas notas -----

Entrada, conversão, cálculo e saída formatada em um só roteiro.

```
nome = input("Aluno: ")
n1 = float(input("Nota 1: "))
n2 = float(input("Nota 2: "))

media = (n1 + n2) / 2
situacao = "Aprovado" if media >= 7 else "Reprovado"

print()
print(f'{"RESULTADO":^28}')
print("-" * 28)
print(f'Aluno: <12>{nome}')
print(f'Média: <12>{media:.2f}')
print(f'Situação: <12>{situacao}')
```

--- SAÍDA ---

```
Aluno: Beatriz
Nota 1: 8.5
Nota 2: 6.0
```

RESULTADO

```
-----
Aluno:      Beatriz
Média:      7.25
Situação:   Aprovado
```

.....
>> EXPERIMENTOS PROPOSTOS

- . Peça dois números e imprima soma, diferença, produto e quociente com duas casas decimais.
- . Peça o preço de um produto e imprima o valor com desconto de 12%, alinhado em coluna de largura 12.
- . Reescreva o CÓDIGO 2.9 usando `.format()` e depois usando `%`. Compare a legibilidade.

[3] TIPOS DE DADOS E VARIÁVEIS

Os tipos primitivos, as operações que eles suportam e as armadilhas de conversão que mais aparecem em provas e em laboratório.

Uma variável em Python é um **nome** ligado a um **objeto**. O tipo pertence ao objeto, não ao nome, e por isso o mesmo nome pode apontar para valores de tipos diferentes ao longo do programa.

--- #CÓDIGO 3.1 Os quatro tipos primitivos -----

```
idade = 25          # int
altura = 1.78       # float
nome = "Python"     # str
ativo = True        # bool

print(idade, type(idade))
print(altura, type(altura))
print(nome, type(nome))
print(ativo, type(ativo))
```

```
--- SAÍDA ---
25 <class 'int'>
1.78 <class 'float'>
Python <class 'str'>
True <class 'bool'>
```

--- #CÓDIGO 3.2 Atribuição múltipla e troca de valores -----

```
a, b, c = 1, 2, 3
x = y = z = 100

print(a, b, c, x, y, z)

# Troca sem variavel auxiliar (bem pythonico)
a, b = b, a
print("Depois da troca:", a, b)

# Nome descartavel por convencao
_, sobrenome = "Ramon Martins".split()
print(sobrenome)
```

```
--- SAÍDA ---
1 2 3 100 100 100
Depois da troca: 2 1
Martins
```

Operações numéricas

+-- #CÓDIGO 3.3 Operadores aritméticos -----

```
a, b = 15, 4

print("soma      ", a + b)
print("subtração  ", a - b)
print("multiplicação ", a * b)
print("divisão    ", a / b)    # sempre float
print("divisão inteira", a // b) # descarta a parte fracionaria
print("resto      ", a % b)
print("potência    ", a ** b)
```

--- SAÍDA ---

```
soma      19
subtração  11
multiplicação 60
divisão    3.75
divisão inteira 3
resto      3
potência    50625
```

+-- #CÓDIGO 3.4 Atribuição composta e funções numéricas -----

```
x = 10
x += 5    # x = x + 5
x *= 2    # x = x * 2
x -= 4
print("x =", x)

print(abs(-7.5), round(3.14159, 2), pow(2, 10))
print(min(3, 9, 1), max(3, 9, 1), sum([3, 9, 1]))
print(divmod(17, 5)) # quociente e resto de uma vez
```

--- SAÍDA ---

```
x = 26
7.5 3.14 1024
1 9 13
(3, 2)
```

[!] PONTO FLUTUANTE NÃO É EXATO

0.1 + 0.2 resulta em 0.30000000000000004. Isso não é um defeito do Python: é a representação binária IEEE 754. Para comparar, use uma tolerância; para dinheiro, use o módulo `decimal`.

+-- #CÓDIGO 3.5 Comparando floats com segurança -----

```
import math

a = 0.1 + 0.2
print(a)
print("a == 0.3?", a == 0.3)
print("próximo o bastante?", math.isclose(a, 0.3))

from decimal import Decimal
print(Decimal("0.1") + Decimal("0.2"))
```

--- SAÍDA ---

```
0.30000000000000004
a == 0.3? False
próximo o bastante? True
0.3
```

+-- #CÓDIGO 3.6 Conversão entre tipos -----

```
print(int("42") + 8)
print(float("19.99"))
print(str(2026) + "/2")
print(int(8.75))      # trunca, nao arredonda
print(round(8.75))    # arredonda
print(int("1010", 2)) # base 2

# Conversao que falha
# int("abc") -> ValueError: invalid literal for int()
```

--- SAÍDA ---

```
50
19.99
2026/2
8
9
10
```

Booleanos e valores verdadeiros

+-- #CÓDIGO 3.7 Booleanos, and, or e not -----

```
nota, frequencia = 8.5, 85

aprovado = nota >= 7 and frequencia >= 75
print("Aprovado:", aprovado)
print("Não aprovado:", not aprovado)
print("Recuperação possível:", nota >= 4 or frequencia >= 90)

# True e False sao inteiros disfarçados
print(True + True, int(False))
```

--- SAÍDA ---

```
Aprovado: True
Não aprovado: False
Recuperação possível: True
2 0
```

+-- #CÓDIGO 3.8 Valores considerados falsos -----

Zero, vazio e `None` são falsos. Todo o resto é verdadeiro. Por isso `if lista:` já testa se a lista tem elementos.

```
vazios = [0, 0.0, "", [], {}, set(), None]
cheios = [1, -3, "texto", [0], {"a": 1}, {0}]

for v in vazios:
    print(f"{repr(v):>8} -> {bool(v)}")

print("---")
print("todos verdadeiros?", all(bool(v) for v in cheios))
```

--- SAÍDA ---

```
0 -> False
0.0 -> False
'' -> False
[] -> False
{} -> False
set() -> False
None -> False
---
todos verdadeiros? True
```

+-- #CÓDIGO 3.9 type() e isinstance() -----

Prefira isinstance(): ele respeita herança e aceita uma tupla de tipos.

```
valor = 3.14

print(type(valor))
print(type(valor) == float)
print(isinstance(valor, float))
print(isinstance(valor, (int, float))) # aceita varios tipos

def dobrar(x):
    if isinstance(x, (int, float)):
        return x * 2
    return "Erro: não é número"

print(dobrar(10), dobrar("abc"))

--- SAÍDA ---
<class 'float'>
True
True
True
20 Erro: não é número
```

+-- #CÓDIGO 3.10 None e constantes -----

Compare com is None, nunca com == None.

```
resultado = None # ausencia de valor
print(resultado, type(resultado))
print("é None?", resultado is None)

# Convencoes de nome (PEP 8)
taxa_juros = 0.05 # snake_case para variaveis
VELOCIDADE_LUZ = 299792458 # MAIUSCULAS para constantes
print(taxa_juros, VELOCIDADE_LUZ)

--- SAÍDA ---
None <class 'NoneType'>
é None? True
0.05 299792458
```

OPERADOR	NOME	EXEMPLO	RESULTADO
+	soma	7 + 2	9
-	subtração	7 - 2	5
*	multiplicação	7 * 2	14
/	divisão real	7 / 2	3.5
//	divisão inteira	7 // 2	3
%	resto	7 % 2	1
**	potência	7 ** 2	49

+-- Tabela 3.1: operadores aritméticos.

>> EXPERIMENTOS PROPOSTOS

- . Peça um número de segundos e converta para horas, minutos e segundos usando // e %.
- . Verifique se `1.1 + 2.2 == 3.3` e corrija a comparação com `math.isclose`.
- . Escreva uma função que receba qualquer valor e devolva o texto "numérico" ou "não numérico".
- . Explique por que `int(-8.7)` e `round(-8.7)` dão resultados diferentes.

=====

|| BLOCO II

CONTROLE DE FLUXO

Decidir caminhos e repetir tarefas sem repetir código.

=====

[4] ESTRUTURAS DE DECISÃO

Fazer o programa escolher um caminho. Em Python isso se resume a `if`, `elif`, `else` e, desde a versão 3.10, `match/case`.

```
[>] FORMA GERAL
if condição:
    bloco executado quando verdadeiro
elif outra_condição:
    bloco alternativo
else:
    bloco quando nada acima valeu
```

- * Dois pontos sempre encerram a linha da condição.
- * O bloco é definido pela indentação, não por chaves.
- * Os operadores lógicos são palavras: `and`, `or`, `not`.
- * `elif` substitui o `else if` de outras linguagens.

+-- #CÓDIGO 4.1 if simples -----

```
a = 10

if a < 20:
    print("a é menor que 20")

print("fim")
```

```
--- SAÍDA ---
a é menor que 20
fim
```

+-- #CÓDIGO 4.2 if e else -----

```
saldo = 100.0
compra = 149.9

if saldo >= compra:
    print("Compra aprovada")
else:
    print(f"Faltam R$ {compra - saldo:.2f}")
```

```
--- SAÍDA ---
Faltam R$ 49.90
```

+-+ #CÓDIGO 4.3 if, elif e else -----

A ordem importa: o primeiro teste verdadeiro vence e os demais nem são avaliados.

```
media = 6.4

if media >= 9:
    conceito = "A"
elif media >= 7:
    conceito = "B"
elif media >= 5:
    conceito = "C"
else:
    conceito = "D"

print(f"Média {media} -> conceito {conceito}")

--- SAÍDA ---
Média 6.4 -> conceito C
```

+-+ #CÓDIGO 4.4 Decisões aninhadas -----

Funciona, mas dois níveis já pedem atenção. O CÓDIGO 4.6 resolve o mesmo caso sem aninhamento.

```
nota = 8.0
frequencia = 60

if nota >= 7:
    if frequencia >= 75:
        print("Aprovado")
    else:
        print("Reprovado por frequência")
else:
    print("Reprovado por nota")

--- SAÍDA ---
Reprovado por frequência
```

OPERADOR	SIGNIFICADO	EXEMPLO VERDADEIRO
==	igual a	3 == 3
!=	diferente de	3 != 4
>	maior que	4 > 3
<	menor que	3 < 4
>=	maior ou igual	3 >= 3
<=	menor ou igual	3 <= 4
in	pertence a	"a" in "casa"
is	é o mesmo objeto	x is None

+-+ Tabela 4.1: operadores de comparação.

```
[!] UM SINAL DE IGUAL NÃO COMPARA
= atribui, == compara. Escrever if x = 10: gera SyntaxError. Python protege você desse erro clássico do C.
```

+-+ #CÓDIGO 4.5 Operadores lógicos -----

```
nota, frequencia = 8.0, 60

if nota >= 7 and frequencia >= 75:
    print("Aprovado")
elif nota >= 7 and frequencia < 75:
    print("Reprovado por frequência")
else:
    print("Reprovado por nota")

print("Tem alguma pendência?", not (nota >= 7 and frequencia >= 75))

--- SAÍDA ---
Reprovado por frequência
Tem alguma pendência? True
```

+-+ #CÓDIGO 4.6 Comparações encadeadas -----

Escrever `18 <= idade < 60` é mais claro do que `idade >= 18 and idade < 60`.

```
idade = 25
temperatura = 36.8

# Em Python isso é legal e legível:
if 18 <= idade < 60:
    print("Faixa adulta")

if 36.0 <= temperatura <= 37.5:
    print("Temperatura normal")

--- SAÍDA ---
Faixa adulta
Temperatura normal
```

+-+ #CÓDIGO 4.7 in e not in -----

Funciona em strings, listas, tuplas, conjuntos e dicionários (nas chaves).

```
vogais = "aeiou"
letra = "e"
print(letra in vogais)

turma = ["Ana", "Bruno", "Carlos"]
print("Diana" not in turma)

if letra in vogais and "Ana" in turma:
    print("As duas verificações passaram")

--- SAÍDA ---
True
True
As duas verificações passaram
```

+-- #CÓDIGO 4.8 Operador ternário -----

Use apenas quando a linha continuar legível. Encadear ternários costuma piorar o código.

```
idade = 17
status = "maior" if idade >= 18 else "menor"
print(f"Você é {status} de idade")

nota = 6.5
print("Aprovado" if nota >= 7 else "Recuperação")

--- SAÍDA ---
Você é menor de idade
Recuperação
```

+-- #CÓDIGO 4.9 match/case (Python 3.10+) -----

Mais do que um switch: match compara estruturas e já extrai valores.

```
def descrever(comando):
    match comando.split():
        case ["ligar"]:
            return "Ligando o equipamento"
        case ["desligar"]:
            return "Desligando"
        case ["canal", numero]:
            return f"Mudando para o canal {numero}"
        case _:
            return "Comando desconhecido"

for c in ["ligar", "canal 12", "reiniciar"]:
    print(f"{c:12} -> {descrever(c)}")

--- SAÍDA ---
ligar      -> Ligando o equipamento
canal 12   -> Mudando para o canal 12
reiniciar  -> Comando desconhecido
```

+-- #CÓDIGO 4.10 Menu com condicionais -----

```
print("=== MENU ===")
print("1 - Cadastrar")
print("2 - Consultar")
print("0 - Sair")

opcao = input("Escolha: ")

if opcao == "1":
    print("Cadastro selecionado")
elif opcao == "2":
    print("Consulta selecionada")
elif opcao == "0":
    print("Encerrando")
else:
    print("Opção inválida")

--- SAÍDA ---
=== MENU ===
1 - Cadastrar
2 - Consultar
0 - Sair
Escolha: 2
Consulta selecionada
```

+-- #CÓDIGO 4.11 Validando o que foi digitado -----

```
entrada = input("Digite algo: ")

if entrada.isdigit():
    numero = int(entrada)
    tipo = "par" if numero % 2 == 0 else "impar"
    print(f"Número inteiro {tipo}")
elif entrada.replace(".", "", 1).isdigit():
    print("Número com casas decimais")
elif entrada.isalpha():
    print("Somente letras")
else:
    print("Combinação de caracteres")
```

--- SAÍDA ---

```
Digite algo: 144
Número inteiro par
```

.....
>> EXPERIMENTOS PROPOSTOS

- . Peça três notas e informe o conceito (A a D) usando `elif`.
- . Escreva um verificador de senha: mínimo 8 caracteres, com pelo menos um dígito.
- . Refaça o CÓDIGO 4.4 sem aninhamento, usando `and`.
- . Converta o menu do CÓDIGO 4.11 para `match/case`.

[5] ESTRUTURAS DE REPETIÇÃO

`for` percorre coleções, `while` repete enquanto uma condição for verdadeira. Saber escolher entre os dois evita metade dos laços infinitos.

* **for**: use quando a quantidade de repetições é conhecida ou quando existe algo a percorrer.

* **while**: use quando a repetição depende de um evento, como uma resposta do usuário.

* **break** encerra o laço, **continue** pula para a próxima volta.

* Python não tem `++`; use `+= 1`.

--- #CÓDIGO 5.1 `for` com `range()` -----
`range(5)` gera 0, 1, 2, 3, 4. O fim é sempre exclusivo.

```
for i in range(5):
    print("volta", i)
```

--- SAÍDA ---

```
volta 0
volta 1
volta 2
volta 3
volta 4
```

--- #CÓDIGO 5.2 `range` com início, fim e passo -----

```
print(list(range(10, 15)))
print(list(range(0, 21, 5)))
print(list(range(10, 0, -2)))
```

```
for i in range(1, 6):
    print(f"{i} ao quadrado = {i ** 2}")
```

--- SAÍDA ---

```
[10, 11, 12, 13, 14]
[0, 5, 10, 15, 20]
[10, 8, 6, 4, 2]
1 ao quadrado = 1
2 ao quadrado = 4
3 ao quadrado = 9
4 ao quadrado = 16
5 ao quadrado = 25
```

--- #CÓDIGO 5.3 Percorrendo uma lista -----

Percorra os elementos diretamente. Usar `range(len(lista))` só é necessário quando você precisa alterar a lista pelo índice.

```
frutas = ["maçã", "banana", "laranja"]
```

```
for fruta in frutas:
    print(f"- {fruta} ({len(fruta)} letras)")
```

--- SAÍDA ---

```
- maçã (4 letras)
- banana (6 letras)
- laranja (7 letras)
```

+-- #CÓDIGO 5.4 Percorrendo texto e contando -----

```
palavra = "engenharia"
vogais = 0

for letra in palavra:
    if letra in "aeiou":
        vogais += 1

print(f"{palavra} tem {vogais} vogais")
```

```
--- SAÍDA ---
engenharia tem 5 vogais
```

+-----

+-- #CÓDIGO 5.5 enumerate(): índice e valor -----

```
cores = ["vermelho", "azul", "verde"]

for i, cor in enumerate(cores):
    print(i, cor)

print("---")
for posicao, cor in enumerate(cores, start=1):
    print(f"{posicao}º lugar: {cor}")
```

```
--- SAÍDA ---
0 vermelho
1 azul
2 verde
---
1º lugar: vermelho
2º lugar: azul
3º lugar: verde
```

+-----

+-- #CÓDIGO 5.6 zip(): duas listas ao mesmo tempo -----

zip para na menor das sequências.

```
alunos = ["Ana", "Bruno", "Carlos"]
notas = [9.5, 7.0, 8.2]

for aluno, nota in zip(alunos, notas):
    print(f"{aluno:<8} {nota:>4.1f}")

print(dict(zip(alunos, notas)))
```

```
--- SAÍDA ---
Ana      9.5
Bruno    7.0
Carlos   8.2
{'Ana': 9.5, 'Bruno': 7.0, 'Carlos': 8.2}
```

+-- #CÓDIGO 5.7 while básico -----

Esqueça o `i += 1` e o laço nunca termina. Interrompa com Ctrl+C.

```
i = 0
while i < 5:
    print("valor:", i)
    i += 1

print("fim do laço")
```

```
--- SAÍDA ---
valor: 0
valor: 1
valor: 2
valor: 3
valor: 4
fim do laço
```

+-- #CÓDIGO 5.8 while com acumulador -----

```
numero = 1
soma = 0

while numero <= 10:
    soma += numero
    numero += 1

print("soma de 1 a 10:", soma)
print("conferindo:", sum(range(1, 11)))
```

```
--- SAÍDA ---
soma de 1 a 10: 55
conferindo: 55
```

+-- #CÓDIGO 5.9 break: interrompendo o laço -----

```
for i in range(1, 100):
    if i * i > 50:
        print(f"o primeiro quadrado acima de 50 é {i * i}")
        break
    print(i * i, end=" ")
```

```
--- SAÍDA ---
1 4 9 16 25 36 49 o primeiro quadrado acima de 50 é 64
```

+-- #CÓDIGO 5.10 continue: pulando uma volta -----

```
for i in range(1, 11):
    if i % 3 == 0:
        continue
    print(i, end=" ")

print("\n(múltiplos de 3 foram pulados)")
```

```
--- SAÍDA ---
1 2 4 5 7 8 10
(múltiplos de 3 foram pulados)
```

+-- #CÓDIGO 5.11 for com else -----

O `else` do laço executa apenas se o laço terminar `sem break`. É a forma limpa de dizer "não encontrei".

```
nomes = ["Ana", "Bruno", "Carlos"]
procurado = "Diana"

for nome in nomes:
    if nome == procurado:
        print("encontrado")
        break
    else:
        print(f"{procurado} não está na lista")
```

```
--- SAÍDA ---
Diana não está na lista
```

+-----

+-- #CÓDIGO 5.12 Laço controlado por menu -----

`while True` com `break` é o padrão para menus.

```
while True:
    print("\n1 - Somar | 2 - Sobre | 0 - Sair")
    opcao = input("Opção: ")

    if opcao == "1":
        a = float(input("a: "))
        b = float(input("b: "))
        print("resultado:", a + b)
    elif opcao == "2":
        print("Manual de Experimentos com Python")
    elif opcao == "0":
        print("Encerrando")
        break
    else:
        print("Opção inválida")
```

```
--- SAÍDA ---
1 - Somar | 2 - Sobre | 0 - Sair
Opção: 1
a: 2
b: 3
resultado: 5.0

1 - Somar | 2 - Sobre | 0 - Sair
Opção: 2
Manual de Experimentos com Python

1 - Somar | 2 - Sobre | 0 - Sair
Opção: 0
Encerrando
```

+-----

+-- #CÓDIGO 5.13 Laços aninhados: tabuada -----

O laço interno completa todas as voltas a cada volta do externo.

```
for i in range(1, 4):
    print(f"\nTabuada do {i}")
    for j in range(1, 6):
        print(f"  {i} x {j} = {i * j}")
```

--- SAÍDA ---

```
Tabuada do 1
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
```

```
Tabuada do 2
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
```

```
Tabuada do 3
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
```

+-- #CÓDIGO 5.14 Validação robusta de entrada -----

Combinação de `while`, `try` e `break` que aparece em quase todo programa interativo.

```
while True:
    try:
        idade = int(input("Idade (0 a 120): "))
    except ValueError:
        print("Digite apenas números inteiros")
        continue

    if 0 <= idade <= 120:
        break
    print("Fora da faixa permitida")

print("Idade registrada:", idade)
```

--- SAÍDA ---

```
Idade (0 a 120): abc
Digite apenas números inteiros
Idade (0 a 120): 999
Fora da faixa permitida
Idade (0 a 120): 38
Idade registrada: 38
```

[*] CONTADOR, ACUMULADOR E BANDEIRA

Três papéis se repetem em quase todo laço: o **contador** (`total += 1`), o **acumulador** (`soma += valor`) e a **bandeira** (`encontrou = True`). Identificá-los facilita escrever o laço certo na primeira tentativa.

>> EXPERIMENTOS PROPOSTOS

- Some apenas os números pares entre 1 e 100 usando `for`.
- Peça números ao usuário até que ele digite 0 e informe a média dos valores digitados.
- Imprima a tabuada completa de 1 a 10 em colunas alinhadas.
- Escreva um jogo de adivinhação: o programa sorteia um número e o usuário tem 5 tentativas.

=====

|| BLOCO III

COLEÇÕES E TEXTO

Guardar muitos valores, percorrê-los e transformá-los.

=====

[6] ESTRUTURAS DE DADOS

Lista, tupla, dicionário e conjunto. Escolher a estrutura certa costuma simplificar mais o programa do que qualquer truque de sintaxe.

ESTRUTURA	SINTAXE	ORDENADA	MUTÁVEL	DUPLICATAS	USO TÍPICO
Lista	[1, 2, 3]	sim	sim	sim	sequência que muda
Tupla	(1, 2, 3)	sim	não	sim	registro fixo
Dicionário	{"a": 1}	sim	sim	chaves únicas	associar chave a valor
Conjunto	{1, 2, 3}	não	sim	não	eliminar repetidos

+-- Tabela 6.1: mapa rápido das quatro estruturas.

Listas

+-- #CÓDIGO 6.1 Criando e acessando -----

```
frutas = ["maçã", "banana", "laranja"]
mista = ["Python", 3.12, True, None]

print(frutas[0])    # primeiro
print(frutas[-1])   # último
print(len(frutas), len(mista))
```

--- SAÍDA ---

```
maçã
laranja
3 4
```

+-- #CÓDIGO 6.2 Adicionando e removendo -----

```
animais = ["gato", "cachorro"]

animais.append("pássaro")    # no fim
animais.insert(1, "peixe")    # na posicao 1
print(animais)

animais.remove("gato")        # por valor
ultimo = animais.pop()        # remove e devolve o ultimo
print(ultimo, animais)

animais.extend(["arara", "tatu"])
print(animais)
```

--- SAÍDA ---

```
['gato', 'peixe', 'cachorro', 'pássaro']
pássaro ['peixe', 'cachorro']
['peixe', 'cachorro', 'arara', 'tatu']
```

+-- #CÓDIGO 6.3 Informações e busca -----

```
numeros = [12, 5, 8, 5, 20]

print("tamanho:", len(numeros))
print("soma:", sum(numeros))
print("maior/menor:", max(numeros), min(numeros))
print("média:", sum(numeros) / len(numeros))
print("o 5 aparece", numeros.count(5), "vezes")
print("posição do 8:", numeros.index(8))
print("20 está na lista?", 20 in numeros)
```

--- SAÍDA ---

```
tamanho: 5
soma: 50
maior/menor: 20 5
média: 10.0
o 5 aparece 2 vezes
posição do 8: 2
20 está na lista? True
```

+-- #CÓDIGO 6.4 Ordenando -----

`sorted()` devolve uma nova lista; `.sort()` modifica a original e devolve `None`.

```
notas = [7.5, 9.0, 5.5, 8.0]

print(sorted(notas))           # copia ordenada
print(sorted(notas, reverse=True))
print("original intacta:", notas)

notas.sort()                   # ordena no lugar
print("depois de sort():", notas)

nomes = ["carlos", "Ana", "bruno"]
print(sorted(nomes, key=str.lower))
```

--- SAÍDA ---

```
[5.5, 7.5, 8.0, 9.0]
[9.0, 8.0, 7.5, 5.5]
original intacta: [7.5, 9.0, 5.5, 8.0]
depois de sort(): [5.5, 7.5, 8.0, 9.0]
['Ana', 'bruno', 'carlos']
```

+-- #CÓDIGO 6.5 Fatiamento (slicing) -----

A forma geral é `[início:fim:passo]`, com o fim sempre exclusivo.

```
letras = ["a", "b", "c", "d", "e", "f", "g"]

print(letras[:3])             # início ate 2
print(letras[-3:])            # últimos tres
print(letras[2:5])            # do meio
print(letras[::2])            # de dois em dois
print(letras[::-1])           # invertida
```

--- SAÍDA ---

```
['a', 'b', 'c']
['e', 'f', 'g']
['c', 'd', 'e']
['a', 'c', 'e', 'g']
['g', 'f', 'e', 'd', 'c', 'b', 'a']
```

+-- #CÓDIGO 6.6 Cópia de listas: a armadilha da referência -----

Atribuir uma lista a outro nome não copia nada. É o erro silencioso mais comum de quem está começando.

```
original = [1, 2, 3]
apelido = original      # mesmo objeto
copia = original.copy() # objeto novo (ou original[:])

apelido.append(99)

print("original:", original)
print("apelido:", apelido)
print("cópia:", copia)
print("apelido é o mesmo objeto?", apelido is original)
```

--- SAÍDA ---

```
original: [1, 2, 3, 99]
apelido: [1, 2, 3, 99]
cópia: [1, 2, 3]
apelido é o mesmo objeto? True
```

+-- #CÓDIGO 6.7 Listas de listas (matriz) -----

```
matriz = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9],
]

print(matriz[1][2])      # linha 1, coluna 2

for linha in matriz:
    print(" ".join(f"{v:3d}" for v in linha))

diagonal = [matriz[i][i] for i in range(3)]
print("diagonal:", diagonal)
```

--- SAÍDA ---

```
6
 1  2  3
 4  5  6
 7  8  9
diagonal: [1, 5, 9]
```

Tuplas

+-- #CÓDIGO 6.8 Criando tuplas -----

(42) é apenas o número 42 entre parênteses. A vírgula é o que cria a tupla.

```
coordenada = (10, 20)
cores = ("vermelho", "azul", "verde")
sozinha = (42,)      # a virgula e obrigatoria
sem_parenteses = 1, 2, 3

print(coordenada[0], cores[-1], sozinho, sem_parenteses)
print(type(sozinha), type((42)))
```

--- SAÍDA ---

```
10 verde (42,) (1, 2, 3)
<class 'tuple'> <class 'int'>
```

+-- #CÓDIGO 6.9 Imutabilidade na prática -----

```
cores = ("vermelho", "azul")

try:
    cores[0] = "amarelo"
except TypeError as erro:
    print("Erro:", erro)

# Para "mudar", crie outra tupla
cores = cores + ("verde",)
print(cores)

--- SAÍDA ---
Erro: 'tuple' object does not support item assignment
('vermelho', 'azul', 'verde')
```

+-- #CÓDIGO 6.10 Desempacotamento -----

```
ponto = (15, 25)
x, y = ponto
print(x, y)

pessoa = ("Ramon", 38, "São José")
nome, idade, cidade = pessoa
print(f"{nome}, {idade}, {cidade}")

# O asterisco recolhe o resto
primeiro, *meio, ultimo = [1, 2, 3, 4, 5]
print(primeiro, meio, ultimo)

--- SAÍDA ---
15 25
Ramon, 38, São José
1 [2, 3, 4] 5
```

Dicionários

+-- #CÓDIGO 6.11 Criando e acessando -----

[] gera `KeyError` se a chave não existir; `.get()` devolve `None` ou o padrão que você escolher.

```
aluno = {
    "nome": "Maria",
    "curso": "Telecomunicações",
    "fase": 4,
}

print(aluno["nome"])
print(aluno.get("fase"))
print(aluno.get("email", "não informado")) # valor padrao

--- SAÍDA ---
Maria
4
não informado
```

+-- #CÓDIGO 6.12 Modificando -----

```
estudante = {"nome": "Ana", "curso": "Python"}

estudante["fase"] = 2          # adiciona
estudante["curso"] = "Ciência de Dados" # atualiza
estudante.update({"nota": 9.5, "ativo": True})
print(estudante)

nota = estudante.pop("nota")    # remove e devolve
del estudante["ativo"]          # remove
print(nota, estudante)

--- SAÍDA ---
{'nome': 'Ana', 'curso': 'Ciência de Dados', 'fase': 2, 'nota': 9.5, 'ativo': True}
9.5 {'nome': 'Ana', 'curso': 'Ciência de Dados', 'fase': 2}
```

+-- #CÓDIGO 6.13 Percorrendo -----

```
notas = {"João": 8.5, "Maria": 9.0, "Pedro": 7.5}

for nome in notas:          # percorre as chaves
    print(nome, end=" ")
print()

for nota in notas.values():
    print(nota, end=" ")
print()

for nome, nota in notas.items(): # forma mais usada
    print(f"{nome:<8} {nota:>4.1f}")

--- SAÍDA ---
João Maria Pedro
8.5 9.0 7.5
João      8.5
Maria     9.0
Pedro     7.5
```

+-- #CÓDIGO 6.14 Contando ocorrências com dicionário -----

O padrão `d[k] = d.get(k, 0) + 1` resolve qualquer contagem. A seção 14 mostra a versão com `Counter`.

```
texto = "engenharia de telecomunicacoes"
contagem = {}

for letra in texto:
    if letra == " ":
        continue
    contagem[letra] = contagem.get(letra, 0) + 1

mais_comuns = sorted(contagem.items(),
                     key=lambda par: par[1],
                     reverse=True)
print(mais_comuns[:5])

--- SAÍDA ---
[('e', 6), ('n', 3), ('a', 3), ('c', 3), ('i', 2)]
```

+-- #CÓDIGO 6.15 Dicionários aninhados -----

```
turma = {
    "Ana": {"notas": [8.5, 9.0], "faltas": 2},
    "Bruno": {"notas": [7.0, 6.0], "faltas": 6},
}

for nome, dados in turma.items():
    notas = dados["notas"]
    media = sum(notas) / len(notas)
    print(f"{nome:<6} média {media:.1f} faltas {dados['faltas']}")
```

--- SAÍDA ---

```
Ana      média 8.8 faltas 2
Bruno    média 6.5 faltas 6
```

Conjuntos

+-- #CÓDIGO 6.16 Criando e eliminando repetidos -----

```
frutas = {"maçã", "banana", "maçã"}
print(frutas)

numeros = [1, 2, 2, 3, 3, 3, 4]
print(set(numeros))
print("sem repetidos, ordenado:", sorted(set(numeros)))

vazio = set()      # cuidado: {} cria um dicionario
print(type(vazio), type({}))
```

--- SAÍDA ---

```
{'maçã', 'banana'}
{1, 2, 3, 4}
sem repetidos, ordenado: [1, 2, 3, 4]
<class 'set'> <class 'dict'>
```

+-- #CÓDIGO 6.17 Adicionando e removendo -----

```
cores = {"vermelho", "azul"}

cores.add("amarelo")
cores.update(["verde", "roxo"])
cores.discard("inexistente")  # nao gera erro
cores.remove("roxo")          # geraria KeyError se faltasse

print(sorted(cores))
```

--- SAÍDA ---

```
['amarelo', 'azul', 'verde', 'vermelho']
```

+-- #CÓDIGO 6.18 Operações de conjunto -----

Também existem os métodos `.union()`, `.intersection()`, `.difference()` e `.symmetric_difference()`.

```
turma_a = {"Ana", "Bruno", "Carlos", "Diana"}
turma_b = {"Bruno", "Diana", "Eduardo"}

print("união:      ", sorted(turma_a | turma_b))
print("interseção:  ", sorted(turma_a & turma_b))
print("só na turma A:", sorted(turma_a - turma_b))
print("exclusivos:  ", sorted(turma_a ^ turma_b))

--- SAÍDA ---
união:      ['Ana', 'Bruno', 'Carlos', 'Diana', 'Eduardo']
interseção:  ['Bruno', 'Diana']
só na turma A: ['Ana', 'Carlos']
exclusivos:  ['Ana', 'Carlos', 'Eduardo']
```

+-- #CÓDIGO 6.19 Verificações entre conjuntos -----

```
vogais = {"a", "e", "i", "o", "u"}
algumas = {"a", "e"}
consoantes = {"x", "y", "z"}

print("a é vogal?", "a" in vogais)
print("subconjunto?", algumas.issubset(vogais))
print("superconjunto?", vogais.issuperset(algumas))
print("sem elementos comuns?", vogais.isdisjoint(consoantes))

--- SAÍDA ---
a é vogal? True
subconjunto? True
superconjunto? True
sem elementos comuns? True
```

Pilha e fila

+-- #CÓDIGO 6.20 Lista como pilha, deque como fila -----

`lista.pop(0)` também funciona como fila, mas é lento em listas grandes porque desloca todos os elementos.

```
from collections import deque

# Pilha: ultimo a entrar, primeiro a sair
pilha = []
pilha.append("página 1")
pilha.append("página 2")
print("desempilhou:", pilha.pop(), "| resta:", pilha)

# Fila: primeiro a entrar, primeiro a sair
fila = deque(["cliente 1", "cliente 2"])
fila.append("cliente 3")
print("atendeu:", fila.popleft(), "| resta:", list(fila))

--- SAÍDA ---
desempilhou: página 2 | resta: ['página 1']
atendeu: cliente 1 | resta: ['cliente 2', 'cliente 3']
```

```
registros = ["ok", "erro", "ok", "ok", "erro"]

print("lista mantém a ordem e as repetições:", registros)
print("conjunto responde 'quais tipos existem':", set(registros))

resumo = {}
for r in registros:
    resumo[r] = resumo.get(r, 0) + 1
print("dicionário responde 'quantos de cada':", resumo)

print("tupla protege o que não muda:", tuple(sorted(set(registros))))

--- SAÍDA ---

lista mantém a ordem e as repetições: ['ok', 'erro', 'ok', 'ok', 'erro']
conjunto responde 'quais tipos existem': {'ok', 'erro'}
dicionário responde 'quantos de cada': {'ok': 3, 'erro': 2}
tupla protege o que não muda: ('erro', 'ok')
```

.....
>> EXPERIMENTOS PROPOSTOS

- . Leia 5 números, guarde em uma lista e informe maior, menor e média.
- . Dadas duas listas de alunos, descubra quem está nas duas usando conjuntos.
- . Monte um dicionário de agenda (nome para telefone) com opções de incluir, consultar e remover.
- . Explique por que `a = b` e `a = b.copy()` produzem comportamentos diferentes ao alterar a lista.

[7] COMPREENSÕES

A forma idiomática de criar listas, dicionários e conjuntos a partir de outra coleção. Substitui laços de três ou quatro linhas por uma expressão legível.

[>] FORMA GERAL

[expressão for item in coleção if condição]

Leia da esquerda para a direita: a expressão aplicada a cada item da coleção que satisfaz a condição.

+-- #CÓDIGO 7.1 Da repetição para a compreensão -----

```
# Versao com laço
quadrados = []
for n in range(1, 6):
    quadrados.append(n ** 2)
print(quadrados)

# Mesma coisa em uma linha
quadrados = [n ** 2 for n in range(1, 6)]
print(quadrados)
```

```
--- SAÍDA ---
[1, 4, 9, 16, 25]
[1, 4, 9, 16, 25]
```

+-- #CÓDIGO 7.2 Compreensão com filtro -----

O if no fim seleciona quais itens entram.

```
numeros = [12, 7, 30, 5, 18, 3]

pares = [n for n in numeros if n % 2 == 0]
grandes = [n for n in numeros if n > 10]

print(pares)
print(grandes)

nomes = ["ana", "Bruno", "carlos"]
print([n.title() for n in nomes if n[0].islower()])
```

```
--- SAÍDA ---
[12, 30, 18]
[12, 30, 18]
['Ana', 'Carlos']
```

+-- #CÓDIGO 7.3 Compreensão com if/else -----

Quando o if/else aparece antes do for, ele escolhe o valor, não filtra a lista.

```
notas = [9.0, 5.5, 7.0, 4.0]

situacao = ["aprovado" if n >= 7 else "reprovado" for n in notas]
print(situacao)

ajustadas = [min(n + 1, 10) if n < 7 else n for n in notas]
print(ajustadas)
```

```
--- SAÍDA ---
['aprovado', 'reprovado', 'aprovado', 'reprovado']
[9.0, 6.5, 7.0, 5.0]
```

+-- #CÓDIGO 7.4 Compreensão aninhada -----

A ordem dos `for` é a mesma dos laços equivalentes, de fora para dentro.

```
matriz = [[1, 2, 3], [4, 5, 6]]

plana = [valor for linha in matriz for valor in linha]
print(plana)

tabela = [[i * j for j in range(1, 5)] for i in range(1, 4)]
for linha in tabela:
    print(linha)
```

```
--- SAÍDA ---
[1, 2, 3, 4, 5, 6]
[1, 2, 3, 4]
[2, 4, 6, 8]
[3, 6, 9, 12]
```

+-- #CÓDIGO 7.5 Compreensão de dicionário -----

```
alunos = ["Ana", "Bruno", "Carlos"]
notas = [9.5, 7.0, 8.2]

boletim = {a: n for a, n in zip(alunos, notas)}
print(boletim)

aprovados = {a: n for a, n in boletim.items() if n >= 8}
print(aprovados)

print({a: round(n) for a, n in boletim.items()})
```

```
--- SAÍDA ---
{'Ana': 9.5, 'Bruno': 7.0, 'Carlos': 8.2}
{'Ana': 9.5, 'Carlos': 8.2}
{'Ana': 10, 'Bruno': 7, 'Carlos': 8}
```

+-- #CÓDIGO 7.6 Compreensão de conjunto -----

```
texto = "engenharia de telecomunicacoes"

letras = {c for c in texto if c.isalpha()}
print(sorted(letras))

tamanhos = {len(p) for p in texto.split()}
print(tamanhos)
```

```
--- SAÍDA ---
['a', 'c', 'd', 'e', 'g', 'h', 'i', 'l', 'm', 'n', 'o', 'r', 's', 't', 'u']
{16, 10, 2}
```

+-- #CÓDIGO 7.7 Expressão geradora -----

Sem colchetes, a expressão produz um item por vez em vez de montar a lista inteira na memória. Para um milhão de elementos, a diferença é enorme.

```
numeros = range(1, 1_000_001)

soma = sum(n for n in numeros if n % 3 == 0)
print("soma dos múltiplos de 3:", soma)

print("algum acima de 999999?", any(n > 999_999 for n in numeros))
print("todos positivos?", all(n > 0 for n in numeros))

--- SAÍDA ---
soma dos múltiplos de 3: 166666833333
algum acima de 999999? True
todos positivos? True
```

+-- #CÓDIGO 7.8 Quando não usar -----

Compreensão é para transformar e filtrar de forma direta. Se a lógica não couber confortavelmente em uma linha, o laço explícito é a escolha melhor.

```
dados = ["12", "abc", "7", "", "30"]

# Difícil de ler em uma linha só:
# validos = [int(d) for d in dados if d.isdigit() and int(d) > 10]

# Preferível quando há mais de uma regra:
validos = []
for d in dados:
    if not d.isdigit():
        continue
    numero = int(d)
    if numero > 10:
        validos.append(numero)

print(validos)

--- SAÍDA ---
[12, 30]
```

>> EXPERIMENTOS PROPOSTOS

- . Crie uma lista com os cubos dos números de 1 a 20 que sejam múltiplos de 3.
- . A partir de uma lista de nomes, monte um dicionário nome para quantidade de letras.
- . Use uma expressão geradora para somar os quadrados de 1 a 1000000 e compare o tempo com a versão em lista.
- . Converta `[[1,2],[3,4],[5,6]]` em `[1,2,3,4,5,6]`.

[8] STRINGS

Texto é o tipo de dado mais manipulado em qualquer programa. Strings em Python são sequências imutáveis: toda operação que parece modificar na verdade cria um texto novo.

+-- #CÓDIGO 8.1 Criando strings -----

```
simples = 'aspas simples'
duplas = "aspas duplas"
mistura = "Ele disse: 'tudo certo'"

bloco = """Primeira linha
Segunda linha"""

vazia = ""
print(simples, "|", duplas, "|", mistura)
print(bloco)
print("string vazia é falsa?", not vazia)
```

--- SAÍDA ---

```
aspas simples | aspas duplas | Ele disse: 'tudo certo'
Primeira linha
Segunda linha
string vazia é falsa? True
```

+-- #CÓDIGO 8.2 Indexação e tamanho -----

```
palavra = "Python"

print(palavra[0], palavra[1], palavra[-1], palavra[-2])
print("tamanho:", len(palavra))

for i, letra in enumerate(palavra):
    print(f"{i:>2} {letra}")
```

--- SAÍDA ---

```
P y n o
tamanho: 6
0 P
1 y
2 t
3 h
4 o
5 n
```

+-- #CÓDIGO 8.3 Fatiamento -----

```
frase = "Python é uma linguagem clara"

print(frase[:6])
print(frase[-5:])
print(frase[7:15])
print(frase[:2])
print(frase[:-1])
```

--- SAÍDA ---

```
Python
clara
é uma li
Pto m igae lr
aralc megaugnil amu é nohtyP
```

+-- #CÓDIGO 8.4 Imutabilidade -----

```
nome = "python"

try:
    nome[0] = "P"
except TypeError as erro:
    print("Erro:", erro)

# A saída é sempre uma string nova
maiuscula = nome.capitalize()
print(nome, "->", maiuscula)

--- SAÍDA ---
Erro: 'str' object does not support item assignment
python -> Python
```

Métodos de transformação

+-- #CÓDIGO 8.5 Caixa e limpeza -----

```
texto = " Python É uma Linguagem incrível "
```

```
print(f"[{texto.strip()}]")
print(f"[{texto.lstrip()}] [{texto.rstrip()}]")
print(texto.strip().upper())
print(texto.strip().lower())
print(texto.strip().title())
print(texto.strip().capitalize())
print("-".join(texto.split()))

--- SAÍDA ---
[Python É uma Linguagem incrível]
[Python É uma Linguagem incrível ] [ Python É uma Linguagem incrível]
PYTHON É UMA LINGUAGEM INCRÍVEL
python é uma linguagem incrível
Python É Uma Linguagem Incrível
Python é uma linguagem incrível
Python-É-uma-Linguagem-incrível
```

+-- #CÓDIGO 8.6 Métodos de verificação -----

Todos devolvem `True` ou `False` e são a base de qualquer validação de entrada.

```
amostras = ["12345", "Python", "Python3", " ", "TITULO"]

for a in amostras:
    print(f"{repr(a):>10} "
          f"digito={a.isdigit():<2} "
          f"alpha={a.isalpha():<2} "
          f"alnum={a.isalnum():<2} "
          f"espaco={a.isspace():<2} "
          f"maiusc={a.isupper():<2}")

--- SAÍDA ---
'12345' digito=1 alpha=0 alnum=1 espaco=0 maiusc=False
'Python' digito=0 alpha=1 alnum=1 espaco=0 maiusc=False
'Python3' digito=0 alpha=0 alnum=1 espaco=0 maiusc=False
' ' digito=0 alpha=0 alnum=0 espaco=1 maiusc=False
'TITULO' digito=0 alpha=1 alnum=1 espaco=0 maiusc=True
```

+-- #CÓDIGO 8.7 Busca dentro do texto -----

.index() faz o mesmo que .find(), mas levanta `ValueError` quando não encontra.

```
frase = "Python é rápido e Python é claro"

print("in:", "rápido" in frase)
print("find:", frase.find("Python"))    # primeira ocorrencia
print("rfind:", frase.rfind("Python"))  # ultima
print("find ausente:", frase.find("Java")) # -1
print("count:", frase.count("Python"))
print("startswith:", frase.startswith("Python"))
print("endswith:", frase.endswith("claro"))
```

```
--- SAÍDA ---
in: True
find: 0
rfind: 18
find ausente: -1
count: 2
startswith: True
endswith: True
```

+-- #CÓDIGO 8.8 Substituição e remoção -----

```
texto = "Eu gosto de Java, Java é interessante"

print(texto.replace("Java", "Python"))
print(texto.replace("Java", "Python", 1)) # so a primeira

telefone = "(48) 99999-1234"
limpo = (telefone.replace("(", "").replace(")", ""))
        .replace(" ", "").replace("-", "")
print("só dígitos:", limpo)
```

```
--- SAÍDA ---
Eu gosto de Python, Python é interessante
Eu gosto de Python, Java é interessante
só dígitos: 489999991234
```

+-+ #CÓDIGO 8.9 split() e join() -----

split transforma texto em lista, join faz o caminho de volta. Note a variável `quebra`: veja a advertência a seguir.

```
linha = "Python,Java,C,Rust"
linguagens = linha.split(",")
print(linguagens)

texto = "palavra1 palavra2\tpalavra3"
print(texto.split())      # qualquer espaço em branco

print("1-2-3-4".split("-", 2)) # no maximo 2 divisoes

frutas = ["maçã", "banana", "uva"]
print(" ".join(frutas))
print(" | ".join(frutas))

quebra = "\n"
print(quebra.join(frutas))
```

```
--- SAÍDA ---
['Python', 'Java', 'C', 'Rust']
['palavra1', 'palavra2', 'palavra3']
['1', '2', '3-4']
maçã, banana, uva
maçã | banana | uva
maçã
banana
uva
```

[!] BARRA INVERTIDA DENTRO DE F-STRING

Até o Python 3.11, escrever `f'{\n'.join(lista)}'` gerava `SyntaxError`. O Python 3.12 passou a aceitar, mas o código deixa de rodar em versões anteriores. A forma portátil é guardar o separador em uma variável antes, como no CÓDIGO 8.9.

Formatação e alinhamento

+-+ #CÓDIGO 8.10 Alinhando colunas -----

```
produtos = [("Cabo HDMI", 39.9), ("Roteador", 289.0),
            ("Antena", 129.5)]

print(f'{"PRODUTO":<14}{"PREÇO":>10}')
print("-" * 24)
for nome, preco in produtos:
    print(f"{nome:<14}{preco:>10.2f}")

print("-" * 24)
print(f'{"TOTAL":<14}{sum(p for _, p in produtos):>10.2f}')
```

```
--- SAÍDA ---
PRODUTO          PREÇO
-----
Cabo HDMI        39.90
Roteador         289.00
Antena           129.50
-----
TOTAL            458.40
```

+-- #CÓDIGO 8.11 Preenchimento e centralização -----

```
titulo = "RELATÓRIO"

print(titulo.center(30, "="))
print(titulo.ljust(20, ".") + "ok")
print(titulo.rjust(30))
print(f"{titulo:*^30}")
print(f"{42:0>5}")
```

```
--- SAÍDA ---

=====RELATÓRIO=====
RELATÓRIO.....ok
                RELATÓRIO
*****RELATÓRIO*****
00042
```

+-----

+-- #CÓDIGO 8.12 Sequências de escape e raw strings -----

O prefixo `r` desliga a interpretação das barras invertidas. Use em caminhos do Windows e em expressões regulares.

```
print("Linha 1\nLinha 2")
print("Coluna1\tColuna2")
print("Aspas: \"Python\"")
print("Barra: C:\\Users\\Documentos")

caminho = r"C:\Users\Documentos\dados.txt"
print(caminho)

padrao = r"\d+\.\d+"      # util em expressoes regulares
print(padrao)
```

```
--- SAÍDA ---

Linha 1
Linha 2
Coluna1 Coluna2
Aspas: "Python"
Barra: C:\Users\Documentos
C:\Users\Documentos\dados.txt
\d+\.\d+
```

+-----

Aplicações

+-- #CÓDIGO 8.13 Extraíndo partes de um endereço -----

```
email = "ramon.mayor@ifsc.edu.br"

usuario, dominio = email.split("@")
print("usuário:", usuario)
print("domínio:", dominio)
print("extensão:", dominio[dominio.rfind("."):])
print("nome formatado:", usuario.replace(".", " ").title())
```

```
--- SAÍDA ---

usuário: ramon.mayor
domínio: ifsc.edu.br
extensão: .br
nome formatado: Ramon Mayor
```

+-----

+-- #CÓDIGO 8.14 Validando um endereço de email -----

```
def validar_email(email):
    """Validação simples: devolve (valido, mensagem)."""
    email = email.strip().lower()

    if not email:
        return False, "não pode estar vazio"
    if email.count("@") != 1:
        return False, "precisa de exatamente um @"

    usuario, dominio = email.split("@")
    if not usuario or not dominio:
        return False, "usuário e domínio são obrigatórios"
    if "." not in dominio:
        return False, "domínio precisa de um ponto"

    return True, "válido"

testes = ["ramon@ifsc.edu.br", "@ifsc.edu.br", "sem-arroba.com",
          " ALUNO@IFSC.EDU.BR "]

for t in testes:
    ok, msg = validar_email(t)
    marca = "ok " if ok else "erro"
    print(f"[{marca}] {t.strip():<24} {msg}")

--- SAÍDA ---
[ok ] ramon@ifsc.edu.br      válido
[erro] @ifsc.edu.br         usuário e domínio são obrigatórios
[erro] sem-arroba.com       precisa de exatamente um @
[ok ] ALUNO@IFSC.EDU.BR     válido
```

+-- #CÓDIGO 8.15 Analisando um texto -----

```
texto = """Python é uma linguagem de programação.
É fácil de aprender e muito poderosa.
Versão usada: 3.12"""

palavras = texto.split()
vogais = sum(1 for c in texto.lower() if c in "aeiou")
digitos = sum(1 for c in texto if c.isdigit())

print("caracteres:", len(texto))
print("palavras:", len(palavras))
print("linhas:", len(texto.splitlines()))
print("vogais:", vogais)
print("dígitos:", digitos)
print("maior palavra:", max(palavras, key=len))

--- SAÍDA ---
caracteres: 95
palavras: 16
linhas: 3
vogais: 30
dígitos: 3
maior palavra: programação.
```

CATEGORIA	MÉTODOS
Caixa	<code>.upper()</code> <code>.lower()</code> <code>.title()</code> <code>.capitalize()</code> <code>.swapcase()</code>
Limpeza	<code>.strip()</code> <code>.lstrip()</code> <code>.rstrip()</code>
Verificação	<code>.isdigit()</code> <code>.isalpha()</code> <code>.isalnum()</code> <code>.isspace()</code> <code>.isupper()</code> <code>.islower()</code>
Busca	<code>.find()</code> <code>.rfind()</code> <code>.index()</code> <code>.count()</code> <code>.startswith()</code> <code>.endswith()</code>
Transformação	<code>.replace()</code> <code>.split()</code> <code>.splitlines()</code> <code>.join()</code>
Alinhamento	<code>.center()</code> <code>.ljust()</code> <code>.rjust()</code> <code>.zfill()</code>

+-- Tabela 8.1: métodos de string mais usados.

.....
>> EXPERIMENTOS PROPOSTOS

- . Peça uma frase e informe quantas palavras e quantas vogais ela tem.
- . Escreva uma função que receba um CPF com pontuação e devolva apenas os dígitos.
- . Monte um cabeçalho de relatório centralizado em 40 colunas com bordas de =.
- . Verifique se uma palavra é palíndromo ignorando maiúsculas e espaços.

=====

|| BLOCO IV

ORGANIZAÇÃO DO CÓDIGO

Funções, módulos e objetos: como um script vira um programa.

=====

[9] FUNÇÕES

Dar nome a um trecho de código é a primeira ferramenta de organização de qualquer programa. Uma função bem escrita faz uma coisa só e o seu nome diz qual é.

```
[>] FORMA GERAL
def nome(parametros):
    """docstring"""
    corpo
    return valor (opcional)
```

+-- #CÓDIGO 9.1 Função sem parâmetros -----

```
def cabecalho():
    """Imprime o cabeçalho padrão do relatório."""
    print("=" * 34)
    print("IFSC - Manual de Experimentos")
    print("=" * 34)

cabecalho()
print("conteúdo do relatório")

--- SAÍDA ---
=====
IFSC - Manual de Experimentos
=====
conteúdo do relatório
```

+-- #CÓDIGO 9.2 Função com parâmetros -----

```
def saudacao(nome, curso):
    print(f"Olá, {nome}! Bem-vindo ao curso de {curso}.")

saudacao("Maria", "Telecomunicações")
saudacao("João", "Redes")
saudacao(curso="Python", nome="Ana") # argumentos nomeados

--- SAÍDA ---
Olá, Maria! Bem-vindo ao curso de Telecomunicações.
Olá, João! Bem-vindo ao curso de Redes.
Olá, Ana! Bem-vindo ao curso de Python.
```

+-- #CÓDIGO 9.3 Retornando um valor -----

`return` encerra a função imediatamente. Sem `return`, a função devolve `None`.

```
def area_retangulo(base, altura):
    return base * altura

def celsius_para_fahrenheit(c):
    return c * 9 / 5 + 32

print(area_retangulo(3, 4))
print(f"{celsius_para_fahrenheit(25):.1f} °F")
print("soma de áreas:", area_retangulo(2, 2) + area_retangulo(3, 3))

--- SAÍDA ---
12
77.0 °F
soma de áreas: 13
```

+-- #CÓDIGO 9.4 Vários valores de retorno -----

Na prática a função devolve uma tupla, que você desempacota na chamada.

```
def estatisticas(valores):
    return min(valores), max(valores), sum(valores) / len(valores)

menor, maior, media = estatisticas([7.5, 9.0, 5.5, 8.0])
print(f"menor {menor} | maior {maior} | média {media:.2f}")

tudo = estatisticas([1, 2, 3])
print(tudo, type(tudo))

--- SAÍDA ---
menor 5.5 | maior 9.0 | média 7.50
(1, 3, 2.0) <class 'tuple'>
```

+-- #CÓDIGO 9.5 Retorno condicional e saída antecipada -----

Tratar os casos inválidos primeiro e sair cedo mantém o corpo da função plano e legível.

```
def classificar(idade):
    if idade < 0:
        return "idade inválida"
    if idade < 13:
        return "criança"
    if idade < 18:
        return "adolescente"
    if idade < 60:
        return "adulto"
    return "idoso"

for i in [-1, 5, 15, 30, 70]:
    print(f"{i:>3} -> {classificar(i)}")

--- SAÍDA ---
-1 -> idade inválida
 5 -> criança
15 -> adolescente
30 -> adulto
70 -> idoso
```

Parâmetros

+-- #CÓDIGO 9.6 Valores padrão -----

Parâmetros com valor padrão vêm sempre depois dos obrigatórios.

```
def criar_perfil(nome, curso="não informado", ativo=True):
    status = "ativo" if ativo else "inativo"
    print(f"{nome:<8} | {curso:<16} | {status}")

criar_perfil("Maria", "Telecomunicações")
criar_perfil("João")
criar_perfil("Ana", ativo=False)

--- SAÍDA ---
Maria   | Telecomunicações | ativo
João    | não informado    | ativo
Ana     | não informado    | inativo
```

+-- #CÓDIGO 9.7 *args: quantidade variável de argumentos -----

```
def somar(*numeros):
    print(f"recebi {numeros} ->", sum(numeros))
    return sum(numeros)

somar(1, 2, 3)
somar(10, 20, 30, 40)
somar()

valores = [5, 10, 15]
somar(*valores)      # o asterisco desempacota a lista

--- SAÍDA ---
recebi (1, 2, 3) -> 6
recebi (10, 20, 30, 40) -> 100
recebi () -> 0
recebi (5, 10, 15) -> 30
```

+-- #CÓDIGO 9.8 **kwargs: argumentos nomeados variáveis -----

```
def relatorio(**dados):
    if not dados:
        print("nenhum dado recebido")
        return
    for chave, valor in dados.items():
        print(f"{chave.replace('_', ' ').title():<12}: {valor}")
    print("-" * 26)

relatorio(nome="Pedro", cargo="Técnico", setor="Redes")
relatorio()

equipamento = {"modelo": "RTL-SDR", "faixa_mhz": "24 a 1766"}
relatorio(**equipamento)

--- SAÍDA ---
Nome      : Pedro
Cargo     : Técnico
Setor     : Redes
-----
nenhum dado recebido
Modelo    : RTL-SDR
Faixa Mhz : 24 a 1766
-----
```

+-- #CÓDIGO 9.9 Combinando todas as formas -----

A ordem é fixa: obrigatórios, com padrão, *args, **kwargs.

```
def registrar(evento, nivel="info", *tags, **extras):
    print(f"[{nivel.upper()}] {evento}")
    if tags:
        print("  tags:", ", ".join(tags))
    if extras:
        print("  extras:", extras)

registrar("sistema iniciado")
registrar("falha na antena", "erro", "rf", "campo", codigo=42)

--- SAÍDA ---
[INFO] sistema iniciado
[ERRO] falha na antena
      tags: rf, campo
      extras: {'codigo': 42}
```

Escopo

+-- #CÓDIGO 9.10 Local e global -----

```
contador = 0          # global

def mostrar():
    interno = 10       # local
    print("dentro:", interno, contador)

mostrar()
print("fora:", contador)

try:
    print(interno)
except NameError as erro:
    print("Erro:", erro)
```

--- SAÍDA ---

```
dentro: 10 0
fora: 0
Erro: name 'interno' is not defined
```

+----- #CÓDIGO 9.11 global e nonlocal -----

Use `global` com parcimônia: funções que dependem de estado externo são mais difíceis de testar.

```
total = 0

def somar_errado(valor):
    total = valor          # cria uma variavel local nova
    return total
```

```
def somar_certo(valor):
    global total
    total += valor
```

```
somar_errado(10)
print("depois do errado:", total)
somar_certo(10)
somar_certo(5)
print("depois do certo:", total)
```

```
def contador():
    n = 0
    def incrementar():
        nonlocal n
        n += 1
        return n
    return incrementar
```

```
c = contador()
print(c(), c(), c())
```

--- SAÍDA ---

```
depois do errado: 0
depois do certo: 15
1 2 3
```

+-+ #CÓDIGO 9.12 Armadilha do parâmetro padrão mutável -----

O valor padrão é criado uma única vez, quando a função é definida. Para listas e dicionários, use `None` como padrão.

```
def adicionar_errado(item, lista=[]):
    lista.append(item)
    return lista

print(adicionar_errado("a"))
print(adicionar_errado("b"))  # a lista lembrou do "a"!

def adicionar_certo(item, lista=None):
    if lista is None:
        lista = []
    lista.append(item)
    return lista

print(adicionar_certo("a"))
print(adicionar_certo("b"))
```

--- SAÍDA ---

```
['a']
['a', 'b']
['a']
['b']
```

Funções como valores

+-+ #CÓDIGO 9.13 lambda e o parâmetro key -----

lambda é uma função anônima de uma expressão só. O uso mais frequente é como `key` em `sorted`, `max` e `min`.

```
quadrado = lambda x: x ** 2
print(quadrado(5))

alunos = [("Ana", 9.5), ("Bruno", 7.0), ("Carlos", 8.2)]

print(sorted(alunos, key=lambda par: par[1]))
print(sorted(alunos, key=lambda par: par[1], reverse=True))
print(max(alunos, key=lambda par: par[1]))
```

--- SAÍDA ---

```
25
[('Bruno', 7.0), ('Carlos', 8.2), ('Ana', 9.5)]
[('Ana', 9.5), ('Carlos', 8.2), ('Bruno', 7.0)]
('Ana', 9.5)
```

+-+ #CÓDIGO 9.14 map, filter e a alternativa com compreensão -----

```
numeros = [1, 2, 3, 4, 5, 6]

print(list(map(lambda x: x ** 2, numeros)))
print(list(filter(lambda x: x % 2 == 0, numeros)))

# Mesma coisa, mais legível para a maioria das pessoas:
print([x ** 2 for x in numeros])
print([x for x in numeros if x % 2 == 0])
```

--- SAÍDA ---

```
[1, 4, 9, 16, 25, 36]
[2, 4, 6]
[1, 4, 9, 16, 25, 36]
[2, 4, 6]
```

+-+ #CÓDIGO 9.15 Passando funções como argumento -----

Funções são objetos: podem ser passadas, guardadas em listas e devolvidas por outras funções.

```
def aplicar(funcao, valores):
    return [funcao(v) for v in valores]

def dobro(x):
    return x * 2

print(aplicar(dobro, [1, 2, 3]))
print(aplicar(str.upper, ["a", "b"]))
print(aplicar(len, ["python", "ifsc"]))
```

```
--- SAÍDA ---
[2, 4, 6]
['A', 'B']
[6, 4]
```

+-+ #CÓDIGO 9.16 Recursão -----

Toda função recursiva precisa de um caso base. Sem ele, o resultado é `RecursionError`.

```
def fatorial(n):
    if n <= 1:          # caso base
        return 1
    return n * fatorial(n - 1)

def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

print([fatorial(i) for i in range(7)])
print([fibonacci(i) for i in range(10)])
```

```
--- SAÍDA ---
[1, 1, 2, 6, 24, 120, 720]
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

+-+ #CÓDIGO 9.17 Documentação e anotações de tipo -----

As anotações não são verificadas em tempo de execução, mas documentam a intenção e alimentam ferramentas como o `mypy` e o autocompletar do editor.

```
def calcular_imc(peso: float, altura: float) -> dict:
    """Calcula o IMC e devolve o valor com a classificação.

    Args:
        peso: massa em quilogramas.
        altura: altura em metros.

    Returns:
        Dicionário com as chaves 'imc' e 'classificacao'.

    Raises:
        ValueError: se peso ou altura não forem positivos.
    """
    if peso <= 0 or altura <= 0:
        raise ValueError("peso e altura devem ser positivos")

    imc = peso / altura ** 2
    if imc < 18.5:
        faixa = "abaixo do peso"
    elif imc < 25:
        faixa = "peso normal"
    elif imc < 30:
        faixa = "sobrepeso"
    else:
        faixa = "obesidade"

    return {"imc": round(imc, 2), "classificacao": faixa}

print(calcular_imc(70, 1.75))
print(calcular_imc.__doc__.splitlines()[0])

--- SAÍDA ---
{'imc': 22.86, 'classificacao': 'peso normal'}
Calcula o IMC e devolve o valor com a classificação.
```

+-+ #CÓDIGO 9.18 Decorador simples -----

Um decorador recebe uma função e devolve outra que a envolve. É assim que frameworks acrescentam comportamento sem alterar o seu código.

```
import time

def cronometrar(funcao):
    """Mede o tempo de execução da função decorada."""
    def interna(*args, **kwargs):
        inicio = time.perf_counter()
        resultado = funcao(*args, **kwargs)
        gasto = time.perf_counter() - inicio
        print(f"{funcao.__name__} levou {gasto:.4f} s")
        return resultado
    return interna

@cronometrar
def somar_ate(n):
    return sum(range(n))

print(somar_ate(1_000_000))

--- SAÍDA ---
somar_ate levou 0.0244 s
499999500000
```

[*] BOAS PRÁTICAS DE FUNÇÃO

Nome no infinitivo dizendo o que ela faz (`calcular_media`, não `calc`). Uma responsabilidade por função. Poucos parâmetros. Evite imprimir e calcular na mesma função: separe o cálculo da apresentação e ela ficará testável.

>> EXPERIMENTOS PROPOSTOS

- . Escreva `media(*notas)` que aceite qualquer quantidade de notas.
- . Crie `maior_de(lista, chave)` que use `key` para achar o maior por qualquer critério.
- . Implemente a soma dos dígitos de um número de forma recursiva.
- . Escreva um decorador `@contar_chamadas` que informe quantas vezes a função foi executada.

[10] MÓDULOS E PACOTES

Todo arquivo `.py` é um módulo. Importar é a forma de reaproveitar o que já foi escrito, seja por você, pela biblioteca padrão ou pela comunidade.

+-- #CÓDIGO 10.1 import de módulo inteiro -----

O nome do módulo fica como prefixo, o que deixa claro de onde a função veio.

```
import math

print(math.pi)
print(math.sqrt(144))
print(math.floor(3.7), math.ceil(3.2))
print(math.log10(1000))
```

```
--- SAÍDA ---
3.141592653589793
12.0
3 4
3.0
```

+-- #CÓDIGO 10.2 from ... import -----

```
from math import pi, sqrt, radians, sin

print(pi)
print(sqrt(81))
print(round(sin(radians(30)), 4))
```

```
# Evite: importa tudo e polui o espaço de nomes
# from math import *
```

```
--- SAÍDA ---
3.141592653589793
9.0
0.5
```

+-- #CÓDIGO 10.3 Apelidos com as -----

Apelidos curtos são convenção em bibliotecas conhecidas, como `import numpy as np`.

```
import statistics as stat
from datetime import datetime as dt

amostras = [12.5, 13.0, 11.8, 12.9]
print("média:", stat.mean(amostras))
print("mediana:", stat.median(amostras))
print("ano atual:", dt(2026, 7, 22).year)
```

```
--- SAÍDA ---
média: 12.55
mediana: 12.7
ano atual: 2026
```

+-+ #CÓDIGO 10.4 Criando o seu próprio módulo -----

Na prática você apenas salva `conversoes.py` na mesma pasta e escreve `import conversoes`.

```
# Aqui o modulo e gravado em disco e depois importado.
codigo = """
def celsius_para_fahrenheit(c):
    return c * 9 / 5 + 32

def watts_para_dbm(p):
    import math
    return 10 * math.log10(p * 1000)
"""

with open("conversoes.py", "w", encoding="utf-8") as arquivo:
    arquivo.write(codigo)

import conversoes

print(conversoes.celsius_para_fahrenheit(25))
print(conversoes.watts_para_dbm(0.001), "dBm")    # 1 mW
print(round(conversoes.watts_para_dbm(0.1), 2), "dBm")

--- SAÍDA ---
77.0
0.0 dBm
20.0 dBm
```

+-+ #CÓDIGO 10.5 O guarda `__name__` -----

Quando o arquivo é executado diretamente, `__name__` vale `"__main__"`. Quando é importado, vale o nome do módulo. Esse teste impede que o código de teste rode ao importar.

```
def principal():
    print("executando o programa")

print("valor de __name__: ", __name__)

if __name__ == "__main__":
    principal()

--- SAÍDA ---
valor de __name__: __main__
executando o programa
```

+-+ #CÓDIGO 10.6 Organização de um projeto -----

Com essa estrutura, `main.py` escreve `from pacote.conversoes import watts_para_dbm`.

```
meu_projeto/
|-- main.py          # ponto de entrada
|-- requisitos.txt   # dependências
|-- dados/
|   `-- medidas.csv
`-- pacote/
    |-- __init__.py  # marca a pasta como pacote
    |-- conversoes.py
    `-- relatorio.py
```

+-- #CÓDIGO 10.7 Ambiente virtual e pip -----

Um ambiente virtual por projeto evita conflito de versões entre trabalhos diferentes.

```
# Criar e ativar um ambiente isolado do sistema
python3 -m venv .venv
source .venv/bin/activate      # Linux e macOS
.venv\Scripts\activate        # Windows

# Instalar, listar e registrar dependências
pip install requests matplotlib
pip list
pip freeze > requisitos.txt

# Reproduzir o ambiente em outra máquina
pip install -r requisitos.txt
```

MÓDULO	SERVE PARA
math	funções matemáticas
random	sorteios e amostragem
datetime	datas e horas
statistics	média, mediana, desvio
collections	contadores e filas
itertools	combinações e permutações
json / csv	leitura e escrita de dados
pathlib / os	caminhos e sistema de arquivos
re	expressões regulares

+-- Tabela 10.1: módulos da biblioteca padrão que mais aparecem no curso.

>> EXPERIMENTOS PROPOSTOS

- . Crie um módulo `geometria.py` com três funções de área e importe-o em outro arquivo.
- . Acrescente ao módulo um bloco `if __name__ == "__main__":` com testes rápidos.
- . Crie um ambiente virtual, instale uma biblioteca e gere o arquivo de requisitos.

[11] PROGRAMAÇÃO ORIENTADA A OBJETOS

Quando dados e comportamento andam juntos, a classe é a forma natural de organizar. Esta seção cobre o essencial para ler e escrever código orientado a objetos.

Uma **classe** é o molde; um **objeto** é cada exemplar criado a partir dele. Os **atributos** guardam o estado e os **métodos** definem o que o objeto sabe fazer.

+-- #CÓDIGO 11.1 Primeira classe -----

`__init__` é chamado automaticamente na criação do objeto. `self` é a referência ao próprio objeto e é sempre o primeiro parâmetro.

```
class Aluno:
    """Representa um aluno da turma."""

    def __init__(self, nome, curso):
        self.nome = nome          # atributo de instancia
        self.curso = curso
        self.notas = []

ana = Aluno("Ana", "Telecomunicações")
bruno = Aluno("Bruno", "Redes")

print(ana.nome, "|", ana.curso, "|", ana.notas)
print(bruno.nome, "|", bruno.curso)
```

--- SAÍDA ---

```
Ana | Telecomunicações | []
Bruno | Redes
```

+-- #CÓDIGO 11.2 Métodos -----

```
class Aluno:
    def __init__(self, nome):
        self.nome = nome
        self.notas = []

    def adicionar_nota(self, nota):
        if 0 <= nota <= 10:
            self.notas.append(nota)
        else:
            print("nota fora da faixa")

    def media(self):
        if not self.notas:
            return 0.0
        return sum(self.notas) / len(self.notas)

    def aprovado(self):
        return self.media() >= 7

ana = Aluno("Ana")
ana.adicionar_nota(8.5)
ana.adicionar_nota(9.0)
ana.adicionar_nota(15)

print(ana.notas, f"média {ana.media():.2f}", ana.aprovado())
```

--- SAÍDA ---

```
nota fora da faixa
[8.5, 9.0] média 8.75 True
```

+-- #CÓDIGO 11.3 __str__ e __repr__ -----

__str__ é para o usuário final e __repr__ é para quem está depurando.

```
class Ponto:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"({self.x}, {self.y})"

    def __repr__(self):
        return f"Ponto(x={self.x}, y={self.y})"

p = Ponto(3, 4)
print(p)          # usa __str__
print(repr(p))    # usa __repr__
print([p, p])     # listas usam __repr__
```

--- SAÍDA ---

```
(3, 4)
Ponto(x=3, y=4)
[Ponto(x=3, y=4), Ponto(x=3, y=4)]
```

+-- #CÓDIGO 11.4 Atributo de classe e atributo de instância -----

```
class Turma:
    instituicao = "IFSC"      # compartilhado por todos
    total = 0

    def __init__(self, codigo):
        self.codigo = codigo # exclusivo de cada objeto
        Turma.total += 1

a = Turma("TEL-2026-1")
b = Turma("RED-2026-1")

print(a.codigo, b.codigo)
print(a.instituicao, b.instituicao)
print("turmas criadas:", Turma.total)
```

--- SAÍDA ---

```
TEL-2026-1 RED-2026-1
IFSC IFSC
turmas criadas: 2
```

+-+ #CÓDIGO 11.5 Encapsulamento com property -----

O sublinhado inicial é um acordo entre programadores: "não mexa aqui de fora". A `property` transforma um método em atributo somente de leitura.

```
class Conta:
    def __init__(self, titular, saldo=0):
        self.titular = titular
        self._saldo = saldo      # convencao: uso interno

    @property
    def saldo(self):
        return self._saldo

    def depositar(self, valor):
        if valor <= 0:
            raise ValueError("valor deve ser positivo")
        self._saldo += valor

    def sacar(self, valor):
        if valor > self._saldo:
            return False
        self._saldo -= valor
        return True

c = Conta("Ana", 100)
c.depositar(50)
print("saque de 500 deu certo?", c.sacar(500))
print("saldo:", c.saldo)

try:
    c.saldo = 1_000_000      # bloqueado: property sem setter
except AttributeError as erro:
    print("Erro:", erro)
```

```
--- SAÍDA ---

saque de 500 deu certo? False
saldo: 150
Erro: property 'saldo' of 'Conta' object has no setter
```

+-+ #CÓDIGO 11.6 Herança -----

```
class Equipamento:
    def __init__(self, modelo):
        self.modelo = modelo
        self.ligado = False

    def ligar(self):
        self.ligado = True
        return f"{self.modelo} ligado"

    def descrever(self):
        return f"Equipamento {self.modelo}"

class Radio(Equipamento):
    def __init__(self, modelo, frequencia):
        super().__init__(modelo)      # reaproveita o __init__
        self.frequencia = frequencia

    def descrever(self):              # sobrescreve
        return f"Rádio {self.modelo} em {self.frequencia} MHz"

r = Radio("RTL-SDR", 145.8)
print(r.ligar())
print(r.descrever())
print("é um Equipamento?", isinstance(r, Equipamento))
```

```
--- SAÍDA ---

RTL-SDR ligado
Rádio RTL-SDR em 145.8 MHz
é um Equipamento? True
```

+++ #CÓDIGO 11.7 Polimorfismo -----

O laço chama `.ganho()` sem saber a classe exata. Cada objeto responde à sua maneira.

```
class Antena:
    def ganho(self):
        return 0.0

class Dipolo(Antena):
    def ganho(self):
        return 2.15

class Yagi(Antena):
    def __init__(self, elementos):
        self.elementos = elementos

    def ganho(self):
        return 2.15 + 2.5 * (self.elementos - 1)

for antena in [Antena(), Dipolo(), Yagi(4)]:
    nome = type(antena).__name__
    print(f"{nome:<8} ganho {antena.ganho():>5.2f} dBi")
```

--- SAÍDA ---

```
Antena  ganho  0.00 dBi
Dipolo   ganho  2.15 dBi
Yagi     ganho  9.65 dBi
```

+++ #CÓDIGO 11.8 dataclass: menos código repetido -----

O decorador gera `__init__`, `__repr__` e `__eq__`. Para atributos mutáveis use `field(default_factory=...)`.

```
from dataclasses import dataclass, field

@dataclass
class Medida:
    sensor: str
    valor: float
    unidade: str = "dBm"
    tags: list = field(default_factory=list)

m1 = Medida("ant-01", -73.5)
m2 = Medida("ant-01", -73.5)

print(m1)
print("iguais?", m1 == m2)      # __eq__ vem de graça
m1.tags.append("teste")
print(m1.tags, m2.tags)
```

--- SAÍDA ---

```
Medida(sensor='ant-01', valor=-73.5, unidade='dBm', tags=[])
iguais? True
['teste'] []
```

```

class Aluno:
    def __init__(self, nome):
        self.nome = nome
        self.notas = []

    def media(self):
        return sum(self.notas) / len(self.notas) if self.notas else 0

class Turma:
    def __init__(self, codigo):
        self.codigo = codigo
        self.alunos = []

    def matricular(self, aluno):
        self.alunos.append(aluno)

    def relatorio(self):
        print(f"Turma {self.codigo}")
        print(f"{'ALUNO':<10}{'MÉDIA':>7} SITUAÇÃO")
        for a in sorted(self.alunos, key=lambda x: -x.media()):
            situacao = "aprovado" if a.media() >= 7 else "exame"
            print(f"{'a.nome':<10}{a.media():>7.2f} {situacao}")

turma = Turma("TEL-2026-1")
for nome, notas in [("Ana", [8.5, 9.0]), ("Bruno", [6.0, 6.5]),
                    ("Carlos", [7.5, 8.0])]:
    aluno = Aluno(nome)
    aluno.notas = notas
    turma.matricular(aluno)

turma.relatorio()

```

--- SAÍDA ---

```

Turma TEL-2026-1
ALUNO      MÉDIA  SITUAÇÃO
Ana         8.75  aprovado
Carlos      7.75  aprovado
Bruno       6.25  exame

```

>> EXPERIMENTOS PROPOSTOS

- . Crie a classe `Retangulo` com métodos `area()` e `perimetro()`.
- . Acrescente `__str__` para que `print(retangulo)` mostre as medidas.
- . Crie `Quadrado` herdando de `Retangulo`.
- . Reescreva a classe `Medida` sem `dataclass` e compare a quantidade de linhas.

=====

|| BLOCO V

DADOS, ROBUSTEZ E PRÁTICA

Persistir informação, tratar falhas e fechar projetos.

=====

[12] MANIPULAÇÃO DE ARQUIVOS

Ler e gravar dados em disco, incluindo os formatos CSV e JSON, que resolvem quase todo intercâmbio de dados em laboratório.

MODO	SIGNIFICADO	SE O ARQUIVO EXISTE	SE NÃO EXISTE
"r"	leitura	lê	erro
"w"	escrita	apaga o conteúdo	cria
"a"	acréscimo	escreve no fim	cria
"x"	criação exclusiva	erro	cria
"rb" / "wb"	modo binário	imagens, áudio	cria

+-- Tabela 12.1: modos de abertura.

[!] O MODO "w" APAGA SEM AVISAR

Abrir um arquivo existente em modo "w" zera o conteúdo no instante da abertura, mesmo que nada seja escrito depois. Para acrescentar informação, use "a".

+-- #CÓDIGO 12.1 Escrevendo um arquivo -----

write não acrescenta quebra de linha: você controla cada \n. Informe sempre o encoding.

```
with open("exemplo.txt", "w", encoding="utf-8") as arquivo:
    arquivo.write("Primeira linha\n")
    arquivo.write("Segunda linha\n")
    arquivo.writelines(["terceira\n", "quarta\n"])

print("arquivo gravado")
```

```
--- SAÍDA ---
arquivo gravado
```

+-- #CÓDIGO 12.2 Lendo de uma vez -----

```
with open("dados.txt", "w", encoding="utf-8") as f:
    f.write("linha 1\nlinha 2\nacentuação: ção, ã, ê\n")

with open("dados.txt", "r", encoding="utf-8") as f:
    conteudo = f.read()

print(conteudo)
print("caracteres lidos:", len(conteudo))
```

```
--- SAÍDA ---
linha 1
linha 2
acentuação: ção, ã, ê
caracteres lidos: 38
```

+-- #CÓDIGO 12.3 Lendo linha a linha -----

Percorrer o arquivo diretamente lê uma linha por vez e funciona com arquivos de qualquer tamanho. `readlines()` carrega tudo na memória.

```
with open("dados.txt", "w", encoding="utf-8") as f:
    f.write("alfa\nbeta\ngama\n")

with open("dados.txt", "r", encoding="utf-8") as f:
    for numero, linha in enumerate(f, 1):
        print(f"{numero}: {linha.strip()}")

with open("dados.txt", "r", encoding="utf-8") as f:
    linhas = f.readlines()
print(linhas)
```

```
--- SAÍDA ---
1: alfa
2: beta
3: gama
['alfa\n', 'beta\n', 'gama\n']
```

+-- #CÓDIGO 12.4 Acrescentando ao final -----

```
with open("registro.log", "w", encoding="utf-8") as f:
    f.write("início da sessão\n")

for evento in ["antena conectada", "sinal capturado"]:
    with open("registro.log", "a", encoding="utf-8") as f:
        f.write(evento + "\n")

with open("registro.log", encoding="utf-8") as f:
    print(f.read())
```

```
--- SAÍDA ---
início da sessão
antena conectada
sinal capturado
```

+-- #CÓDIGO 12.5 with contra abertura manual -----

```
# Forma frágil: se ocorrer um erro antes do close, o arquivo fica aberto
arquivo = open("teste.txt", "w", encoding="utf-8")
arquivo.write("dado")
arquivo.close()
print("fechado manualmente?", arquivo.closed)

# Forma correta: o with fecha ate se houver execucao
with open("teste.txt", "a", encoding="utf-8") as arquivo:
    arquivo.write(" e mais dado")
print("fechado pelo with?", arquivo.closed)
```

```
--- SAÍDA ---
fechado manualmente? True
fechado pelo with? True
```

Caminhos com pathlib

+-- #CÓDIGO 12.6 pathlib: caminhos que funcionam em qualquer sistema -----

pathlib substitui a concatenação de strings e boa parte do módulo `os.path`.

```
from pathlib import Path

pasta = Path("medidas")
pasta.mkdir(exist_ok=True)

arquivo = pasta / "sessao01.txt"      # a barra monta o caminho
arquivo.write_text("-73.5 dBm\n-71.2 dBm\n", encoding="utf-8")

print("existe?", arquivo.exists())
print("nome:", arquivo.name, "| extensão:", arquivo.suffix)
print("tamanho:", arquivo.stat().st_size, "bytes")
print("conteúdo:", arquivo.read_text(encoding="utf-8").split())
print("arquivos na pasta:", [p.name for p in pasta.iterdir()])
```

```
--- SAÍDA ---

existe? True
nome: sessao01.txt | extensão: .txt
tamanho: 20 bytes
conteúdo: ['-73.5', 'dBm', '-71.2', 'dBm']
arquivos na pasta: ['sessao01.txt']
```

Formatos de intercâmbio

+-- #CÓDIGO 12.7 CSV: tabelas -----

`newline=""` evita linhas em branco no Windows. `DictReader` entrega cada linha como dicionário usando o cabeçalho.

```
import csv

linhas = [
    ["sensor", "valor_dbm", "data"],
    ["ant-01", -73.5, "2026-07-22"],
    ["ant-02", -68.1, "2026-07-22"],
]

with open("medidas.csv", "w", newline="", encoding="utf-8") as f:
    csv.writer(f).writerows(linhas)

with open("medidas.csv", encoding="utf-8") as f:
    for registro in csv.DictReader(f):
        print(f"{registro['sensor']}: {registro['valor_dbm']} dBm")
```

```
--- SAÍDA ---

ant-01: -73.5 dBm
ant-02: -68.1 dBm
```

--- #CÓDIGO 12.8 JSON: estruturas aninhadas

dump/load trabalham com arquivos; dumps/loads trabalham com strings. `ensure_ascii=False` preserva os acentos.

```
import json

config = {
    "estacao": "IFSC-389",
    "frequencias": [145.8, 437.5],
    "ativa": True,
    "operador": {"nome": "Ramon", "indicativo": "PU4MAY"},
}

with open("config.json", "w", encoding="utf-8") as f:
    json.dump(config, f, indent=2, ensure_ascii=False)

with open("config.json", encoding="utf-8") as f:
    lido = json.load(f)

print(lido["operador"]["indicativo"])
print(lido["frequencias"][0])
print(json.dumps(lido, ensure_ascii=False)[:60], "...")

--- SAÍDA ---
PU4MAY
145.8
{"estacao": "IFSC-389", "frequencias": [145.8, 437.5], "ativ ...
```

--- #CÓDIGO 12.9 Aplicação: lista de tarefas persistente

Duas funções pequenas, uma para carregar e outra para salvar, resolvem a persistência de quase qualquer trabalho de disciplina.

```
import json
from pathlib import Path

ARQUIVO = Path("tarefas.json")

def carregar():
    if not ARQUIVO.exists():
        return []
    return json.loads(ARQUIVO.read_text(encoding="utf-8"))

def salvar(tarefas):
    ARQUIVO.write_text(
        json.dumps(tarefas, indent=2, ensure_ascii=False),
        encoding="utf-8")

tarefas = carregar()
tarefas.append({"texto": "revisar o manual", "feita": False})
tarefas.append({"texto": "testar a antenna", "feita": True})
salvar(tarefas)

for i, t in enumerate(carregar(), 1):
    marca = "x" if t["feita"] else " "
    print(f"[{marca}] {i}. {t['texto']}")

--- SAÍDA ---
[ ] 1. revisar o manual
[x] 2. testar a antenna
```

>> EXPERIMENTOS PROPOSTOS

- . Grave em arquivo os nomes digitados até o usuário escrever "fim" e depois releia mostrando a numeração.
- . Leia um arquivo de texto e informe quantas linhas, palavras e caracteres ele tem.
- . Converta o CSV do CÓDIGO 12.8 em JSON.
- . Acrescente ao CÓDIGO 12.9 a opção de marcar uma tarefa como concluída.

[13] TRATAMENTO DE EXCEÇÕES

Programas encontram arquivos que não existem, entradas inválidas e divisões por zero. Tratar exceções é decidir o que fazer quando o esperado não acontece.

- * **try**: trecho que pode falhar.
- * **except**: o que fazer quando falha.
- * **else**: executa quando **não** houve erro.
- * **finally**: executa sempre, com erro ou sem erro.
- * **raise**: provoca uma exceção de propósito.

+-- #CÓDIGO 13.1 try e except -----

```
try:
    resultado = 10 / 0
except ZeroDivisionError:
    print("não é possível dividir por zero")
    resultado = None

print("resultado:", resultado)
print("o programa continuou")
```

```
--- SAÍDA ---
não é possível dividir por zero
resultado: None
o programa continuou
```

+-- #CÓDIGO 13.2 Tratando cada erro de um jeito -----

Coloque sempre as exceções específicas antes das genéricas: o primeiro **except** compatível vence.

```
def dividir(texto_a, texto_b):
    try:
        return float(texto_a) / float(texto_b)
    except ZeroDivisionError:
        return "divisor igual a zero"
    except ValueError:
        return "entrada não numérica"

print(dividir("10", "4"))
print(dividir("10", "0"))
print(dividir("dez", "2"))
```

```
--- SAÍDA ---
2.5
divisor igual a zero
entrada não numérica
```

+- #CÓDIGO 13.3 Capturando a mensagem e vários tipos juntos -----

A tupla agrupa tipos que recebem o mesmo tratamento. `as erro` dá acesso à mensagem original.

```
entradas = ["12", "abc", None]

for valor in entradas:
    try:
        print("dobro:", int(valor) * 2)
    except (ValueError, TypeError) as erro:
        print(f"falhou com {valor!r}: {type(erro).__name__}: {erro}")

--- SAÍDA ---
dobro: 24
falhou com 'abc': ValueError: invalid literal for int() with base 10: 'abc'
falhou com None: TypeError: int() argument must be a string, a bytes-like object or a real number, not 'NoneType'
```

+- #CÓDIGO 13.4 else e finally -----

```
def ler_config(nome):
    try:
        arquivo = open(nome, encoding="utf-8")
    except FileNotFoundError:
        print("arquivo não encontrado, usando padrões")
        return {}
    else:
        print("arquivo aberto com sucesso")
        conteudo = arquivo.read()
        return {"linhas": len(conteudo.splitlines())}
    finally:
        print("fim da tentativa de leitura")

with open("config.txt", "w", encoding="utf-8") as f:
    f.write("a=1\nb=2\n")

print(ler_config("config.txt"))
print(ler_config("inexistente.txt"))

--- SAÍDA ---
arquivo aberto com sucesso
fim da tentativa de leitura
{'linhas': 2}
arquivo não encontrado, usando padrões
fim da tentativa de leitura
{}
```

+- #CÓDIGO 13.5 raise: sinalizando um erro -----

Uma função de biblioteca sinaliza o problema com `raise`; quem chamou decide o que fazer.

```
def calcular_media(notas):
    if not notas:
        raise ValueError("a lista de notas está vazia")
    return sum(notas) / len(notas)

print(calcular_media([8, 9, 10]))

try:
    calcular_media([])
except ValueError as erro:
    print("Erro tratado:", erro)

--- SAÍDA ---
9.0
Erro tratado: a lista de notas está vazia
```

+-+ #CÓDIGO 13.6 Exceções próprias -----

Criar uma exceção própria deixa o erro do seu domínio distinguível dos erros da linguagem.

```
class IdadeInvalida(Exception):
    """Erro de domínio: idade fora da faixa aceitável."""

    def __init__(self, idade, mensagem="idade inválida"):
        self.idade = idade
        super().__init__(f"{mensagem} ({idade})")

def validar(idade):
    if not isinstance(idade, int):
        raise TypeError("idade deve ser um inteiro")
    if idade < 0:
        raise IdadeInvalida(idade, "não pode ser negativa")
    if idade > 150:
        raise IdadeInvalida(idade, "acima do limite")
    return True

for candidato in [25, -5, 200, "trinta"]:
    try:
        validar(candidato)
        print(f"{candidato!r}: ok")
    except IdadeInvalida as erro:
        print(f"{candidato!r}: {erro}")
    except TypeError as erro:
        print(f"{candidato!r}: {erro}")
```

--- SAÍDA ---

```
25: ok
-5: não pode ser negativa (-5)
200: acima do limite (200)
'trinta': idade deve ser um inteiro
```

+-+ #CÓDIGO 13.7 Leitura de arquivo com tratamento completo -----

```
def ler_texto(nome):
    try:
        with open(nome, encoding="utf-8") as arquivo:
            conteudo = arquivo.read()
    except FileNotFoundError:
        print(f"'{nome}' não existe")
    except PermissionError:
        print(f"sem permissão para ler '{nome}'")
    except UnicodeDecodeError:
        print(f"'{nome}' não está em UTF-8")
    else:
        if not conteudo.strip():
            print(f"'{nome}' está vazio")
        return conteudo
    return None

with open("ok.txt", "w", encoding="utf-8") as f:
    f.write("conteúdo válido")

print(repr(ler_texto("ok.txt")))
print(repr(ler_texto("faltando.txt")))
```

--- SAÍDA ---

```
'conteúdo válido'
'faltando.txt' não existe
None
```

Lendo um traceback

+-- #CÓDIGO 13.8 Anatomia da mensagem de erro -----

Leia de baixo para cima: a última linha diz o **que** aconteceu, a linha imediatamente acima diz **onde**, e o restante mostra o caminho de chamadas até ali.

```

Traceback (most recent call last):
  File "relatorio.py", line 18, in <module>
    main()
  File "relatorio.py", line 12, in main
    media = calcular(notas)
  File "relatorio.py", line 6, in calcular
    return sum(notas) / len(notas)
           ~~~~~^~~~~~
ZeroDivisionError: division by zero

```

[!] NUNCA CAPTURE TUDO EM SILÊNCIO

except: sozinho ou except Exception: pass engolem inclusive erros de digitação e interrupções do teclado. Se precisar de um tratamento genérico, capture Exception, registre a mensagem e trate apenas o que você sabe tratar.

EXCEÇÃO	CAUSA TÍPICA
ValueError	conversão impossível, como int("abc")
TypeError	operação entre tipos incompatíveis
IndexError	índice fora da lista
KeyError	chave inexistente no dicionário
ZeroDivisionError	divisão por zero
FileNotFoundError	arquivo não existe
PermissionError	sem permissão de acesso
AttributeError	método ou atributo que não existe
NameError	nome usado antes de existir
IndentationError	recuo incorreto

+-- Tabela 13.1: exceções mais frequentes.

-
- >> EXPERIMENTOS PROPOSTOS
- . Peça dois números e trate ValueError e ZeroDivisionError ao dividi-los.
 - . Crie a exceção NotaInvalida e use-a em uma função de cadastro de notas.
 - . Escreva uma função que leia um JSON e trate arquivo ausente e conteúdo malformado.
 - . Provoque cada exceção da Tabela 13.1 e registre a mensagem completa.

[14] BIBLIOTECA PADRÃO ESSENCIAL

Python vem com muita coisa pronta. Conhecer estes módulos evita reescrever o que já existe e testado.

+-- #CÓDIGO 14.1 math: funções matemáticas -----

```
import math

print(math.pi, math.e)
print(math.sqrt(2), math.hypot(3, 4))
print(math.floor(3.7), math.ceil(3.2), math.trunc(-3.7))
print(math.log10(1000), math.log2(1024))
print(math.degrees(math.pi), math.sin(math.radians(90)))
print(math.factorial(5), math.gcd(24, 36))
```

--- SAÍDA ---

```
3.141592653589793 2.718281828459045
1.4142135623730951 5.0
3 4 -3
3.0 10.0
180.0 1.0
120 12
```

+-- #CÓDIGO 14.2 random: sorteios -----

Sem `seed` cada execução dá um resultado diferente. Com `seed`, o sorteio se repete, o que é essencial para testar.

```
import random

random.seed(42)          # torna o resultado reproduzível

print(random.random())
print(random.randint(1, 6))
print(random.uniform(1.5, 2.5))
print(random.choice(["ouro", "prata", "bronze"]))
print(random.sample(range(1, 61), 6))

cartas = ["A", "2", "3", "4"]
random.shuffle(cartas)
print(cartas)
```

--- SAÍDA ---

```
0.6394267984578837
1
2.241550499759833
ouro
[15, 9, 48, 7, 44, 58]
['4', '2', '3', 'A']
```

+-+ #CÓDIGO 14.3 datetime: datas e horas -----

strftime formata para texto e strptime interpreta texto. Use datetime.now() para o instante atual.

```
from datetime import datetime, date, timedelta

hoje = date(2026, 7, 22)
agora = datetime(2026, 7, 22, 14, 30)

print(hoje.strftime("%d/%m/%Y"))
print(agora.strftime("%d/%m/%Y às %H:%M"))
print("dia da semana:", hoje.weekday()) # 0 = segunda

prazo = hoje + timedelta(days=30)
print("prazo:", prazo, "| faltam", (prazo - hoje).days, "dias")

lida = datetime.strptime("01/03/2026", "%d/%m/%Y")
print("convertida:", lida.date())

--- SAÍDA ---
22/07/2026
22/07/2026 às 14:30
dia da semana: 2
prazo: 2026-08-21 | faltam 30 dias
convertida: 2026-03-01
```

+-+ #CÓDIGO 14.4 statistics: medidas resumo -----

```
import statistics as stat

amostras = [12.5, 13.0, 11.8, 12.9, 12.1, 13.4]

print("média:", round(stat.mean(amostras), 3))
print("mediana:", stat.median(amostras))
print("desvio padrão:", round(stat.stdev(amostras), 4))
print("variância:", round(stat.variance(amostras), 4))
print("moda:", stat.mode([1, 2, 2, 3]))

--- SAÍDA ---
média: 12.617
mediana: 12.7
desvio padrão: 0.5981
variância: 0.3577
moda: 2
```

+-+ #CÓDIGO 14.5 collections: Counter e defaultdict -----

Counter resolve contagem em uma linha e defaultdict dispensa verificar se a chave já existe.

```
from collections import Counter, defaultdict, namedtuple

texto = "engenharia de telecomunicacoes"
contagem = Counter(c for c in texto if c.isalpha())
print(contagem.most_common(4))

grupos = defaultdict(list)
for nome in ["Ana", "Bruno", "Alice", "Carlos"]:
    grupos[nome[0]].append(nome)
print(dict(grupos))

Medida = namedtuple("Medida", "sensor valor")
m = Medida("ant-01", -73.5)
print(m, m.sensor, m.valor)

--- SAÍDA ---
[('e', 6), ('n', 3), ('a', 3), ('c', 3)]
{'A': ['Ana', 'Alice'], 'B': ['Bruno'], 'C': ['Carlos']}
Medida(sensor='ant-01', valor=-73.5) ant-01 -73.5
```

+-- #CÓDIGO 14.6 itertools: combinações e agrupamentos -----

```
from itertools import combinations, permutations, product, count

print(list(combinations("ABC", 2)))
print(list(permutations("ABC", 2)))
print(list(product([0, 1], repeat=2)))

contador = count(start=10, step=5)
print([next(contador) for _ in range(4)])

--- SAÍDA ---
[('A', 'B'), ('A', 'C'), ('B', 'C')]
[('A', 'B'), ('A', 'C'), ('B', 'A'), ('B', 'C'), ('C', 'A'), ('C', 'B')]
[(0, 0), (0, 1), (1, 0), (1, 1)]
[10, 15, 20, 25]
```

+-- #CÓDIGO 14.7 re: expressões regulares -----

Use sempre *raw strings* nos padrões. *findall* busca todas as ocorrências, *search* a primeira e *fullmatch* valida o texto inteiro.

```
import re

texto = "Contatos: ramon@ifsc.edu.br, suporte@ifsc.edu.br, (48) 3381-2800"

emails = re.findall(r"[\w.]+\@[w.]+", texto)
print(emails)

telefone = re.search(r"(\d{2})\s?\d{4}-\d{4}", texto)
print(telefone.group() if telefone else "não encontrado")

print(re.sub(r"\d", "#", "senha 12345"))
print(bool(re.fullmatch(r"\d{3}\.\d{3}\.\d{3}-\d{2}",
                        "123.456.789-00")))
```

```
--- SAÍDA ---
['ramon@ifsc.edu.br', 'suporte@ifsc.edu.br']
(48) 3381-2800
senha #####
True
```

+-- #CÓDIGO 14.8 os, sys e time -----

```
import os
import sys
import time

print("versão:", sys.version.split()[0])
print("plataforma:", sys.platform)
print("pasta atual:", os.path.basename(os.getcwd()))
print("variável de ambiente HOME existe?", "HOME" in os.environ)

inicio = time.perf_counter()
soma = sum(range(200_000))
print(f"tempo: {time.perf_counter() - inicio:.4f} s")

--- SAÍDA ---
versão: 3.12.3
plataforma: linux
pasta atual: tmp32igper7
variável de ambiente HOME existe? True
tempo: 0.0049 s
```

.....
>> EXPERIMENTOS PROPOSTOS

- . Sorteie 6 números da megassena sem repetição usando `random.sample`.
- . Calcule quantos dias faltam para o fim do semestre com `timedelta`.
- . Use `Counter` para achar as 10 palavras mais comuns de um arquivo de texto.
- . Escreva uma expressão regular que valide placas de veículo no padrão Mercosul.

[15] ESTILO, DEPURAÇÃO E TESTES

O código é lido muito mais vezes do que é escrito. Esta seção reúne as convenções e as ferramentas que separam um script que funciona de um programa que se pode manter.

PEP 8 em uma página

ELEMENTO	CONVENÇÃO	EXEMPLO
Variável e função	snake_case	<code>calcular_media()</code>
Constante	MAIÚSCULAS	<code>VELOCIDADE_LUZ</code>
Classe	CapWords	<code>ContaCorrente</code>
Módulo	minúsculas curtas	<code>conversoes.py</code>
Indentação	4 espaços	sem tabulação
Linha	até 79 caracteres	quebre dentro de parênteses
Espaços	ao redor de operadores	<code>a = b + 1</code>
Uso interno	sublinhado inicial	<code>_saldo</code>

+-- Tabela 15.1: convenções da PEP 8.

+-- #CÓDIGO 15.1 Antes e depois -----

Mesmo resultado, esforço de leitura bem diferente. Ferramentas como `black`, `ruff` e `flake8` automatizam essa revisão.

```
# Antes
def Calc(L):
    T=0
    for I in range( len(L) ):T=T+L[I]
    return T/len(L)

# Depois
def calcular_media(valores):
    """Retorna a média aritmética de uma sequência numérica."""
    if not valores:
        return 0.0
    return sum(valores) / len(valores)

print(Calc([7, 8, 9]), calcular_media([7, 8, 9]))
```

```
--- SAÍDA ---
8.0 8.0
```

+-- #CÓDIGO 15.2 Anotações de tipo -----

Python não impõe os tipos anotados em tempo de execução, mas o editor e o `mypy` passam a apontar erros antes de você rodar o programa.

```
def media(valores: list[float]) -> float:
    return sum(valores) / len(valores) if valores else 0.0

def formatar(nome: str, nota: float, aprovado: bool = False) -> str:
    marca = "aprovado" if aprovado else "em avaliação"
    return f"{nome}: {nota:.1f} ({marca})"

print(media([8.0, 9.0]))
print(formatar("Ana", 8.5, True))
print(media.__annotations__)
```

```
--- SAÍDA ---
8.5
Ana: 8.5 (aprovado)
{'valores': list[float], 'return': <class 'float'>}
```

Depuração

+-- #CÓDIGO 15.3 assert para verificar hipóteses -----

`assert` documenta e verifica hipóteses durante o desenvolvimento. Não use para validar entrada do usuário: asserções somem quando o Python roda com a opção `-O`.

```
def desconto(preco, percentual):
    assert 0 <= percentual <= 100, "percentual fora da faixa"
    return preco * (1 - percentual / 100)

print(desconto(100, 15))

try:
    desconto(100, 150)
except AssertionError as erro:
    print("AssertionError:", erro)
```

--- SAÍDA ---

```
85.0
AssertionError: percentual fora da faixa
```

+-- #CÓDIGO 15.4 Testes automatizados com unittest -----

Cada método iniciado por `test_` vira um caso de teste. Rodar os testes depois de cada alteração é o que permite refatorar sem medo.

```
import unittest

def media(valores):
    return sum(valores) / len(valores) if valores else 0.0

class TesteMedia(unittest.TestCase):
    def test_lista_normal(self):
        self.assertAlmostEqual(media([8, 9, 10]), 9.0)

    def test_lista_vazia(self):
        self.assertEqual(media([]), 0.0)

    def test_um_elemento(self):
        self.assertEqual(media([7]), 7)

unittest.main(argv=["prog"], exit=False, verbosity=2)
```

--- SAÍDA ---

```
test_lista_normal (__main__.TesteMedia.test_lista_normal) ... ok
test_lista_vazia (__main__.TesteMedia.test_lista_vazia) ... ok
test_um_elemento (__main__.TesteMedia.test_um_elemento) ... ok
```

Ran 3 tests in 0.000s

OK

+-- #CÓDIGO 15.5 logging em vez de print -----

Com logging você liga e desliga o nível de detalhe sem apagar linhas de código, e ainda pode gravar tudo em arquivo.

```
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(levelname)s | %(message)s")

logging.debug("detalhe que só interessa ao desenvolvedor")
logging.info("processando 3 arquivos")
logging.warning("valor de RSSI acima do esperado")
logging.error("falha ao abrir a porta serial")
```

--- SAÍDA ---

```
INFO | processando 3 arquivos
WARNING | valor de RSSI acima do esperado
ERROR | falha ao abrir a porta serial
```

[*] DEPURANDO DE VERDADE

A função `breakpoint()` abre o depurador interativo no ponto em que foi chamada. Comandos úteis: `n` (próxima linha), `s` (entra na função), `c` (continua), `p` *variavel* (imprime) e `q` (sai). No VS Code, o mesmo efeito vem clicando na margem esquerda para marcar a linha.

SINTOMA	CAUSA PROVÁVEL	COMO RESOLVER
<code>IndentationError</code>	espaços e tabulação misturados	configure o editor para 4 espaços
<code>NameError</code>	nome digitado errado ou usado antes de existir	confira a grafia e a ordem
<code>TypeError: can only concatenate str</code>	somar texto com número	converta com <code>str()</code> ou use f-string
Lista muda sozinha	duas variáveis apontam para a mesma lista	use <code>.copy()</code>
Laço nunca termina	condição do <code>while</code> nunca muda	verifique o incremento
Resultado 0 em divisão	divisão inteira com <code>//</code>	use <code>/</code>
Acento vira símbolo	encoding errado na leitura	informe <code>encoding="utf-8"</code>

+-- Tabela 15.2: sintomas frequentes em laboratório.

>> EXPERIMENTOS PROPOSTOS

- . Reescreva um programa antigo seu seguindo a Tabela 15.1.
- . Acrescente anotações de tipo a três funções já escritas.
- . Escreva testes `unittest` para a sua função de média.
- . Troque todos os `print` de depuração de um projeto por chamadas de `logging`.

=====

[16] PROJETOS INTEGRADORES

=====

Quatro programas completos que combinam o conteúdo das seções anteriores. Cada um cabe em uma tela e serve de ponto de partida para trabalhos maiores.

Conjuntos guardam as letras tentadas, `all()` detecta a vitória e uma função pequena cuida da máscara.

```
import random

PALAVRAS = ["python", "antena", "modulacao", "protocolo"]

def mascarar(palavra, achadas):
    return " ".join(c if c in achadas else "_" for c in palavra)

def jogar(tentativas=6):
    random.seed(2) # remove para sortear de fato
    palavra = random.choice(PALAVRAS).upper()
    achadas = set()

    while tentativas > 0:
        print(f"\n{mascarar(palavra, achadas)}  "
              f"({tentativas} tentativas)")

        if all(c in achadas for c in palavra):
            return f"Acertou: {palavra}"

        letra = input("letra: ").upper().strip()
        if len(letra) != 1 or not letra.isalpha():
            print("digite uma única letra")
            continue
        if letra in achadas:
            print("letra já usada")
            continue

        achadas.add(letra)
        if letra in palavra:
            print("existe na palavra")
        else:
            tentativas -= 1
            print("não existe")

    return f"Acabaram as tentativas. Era {palavra}"

print(jogar())
```

```
--- SAÍDA ---
_ _ _ _ _ (6 tentativas)
letra: p
existe na palavra

P _ _ _ _ (6 tentativas)
letra: y
existe na palavra

P Y _ _ _ (6 tentativas)
letra: t
existe na palavra

P Y T _ _ (6 tentativas)
letra: a
não existe

P Y T _ _ (5 tentativas)
letra: h
existe na palavra

P Y T H _ (5 tentativas)
letra: o
existe na palavra

P Y T H O _ (5 tentativas)
letra: n
existe na palavra

P Y T H O N (5 tentativas)
Acertou: PYTHON
```

```

import json
from pathlib import Path

ARQUIVO = Path("boletim.json")

def media(notas):
    return sum(notas) / len(notas) if notas else 0.0

def salvar(turma):
    ARQUIVO.write_text(json.dumps(turma, ensure_ascii=False,
                                  indent=2), encoding="utf-8")

def carregar():
    if ARQUIVO.exists():
        return json.loads(ARQUIVO.read_text(encoding="utf-8"))
    return {}

def relatorio(turma):
    print(f"{'ALUNO':<10}{ 'MÉDIA':>7} SITUAÇÃO")
    print("-" * 30)
    for nome, notas in sorted(turma.items(),
                              key=lambda p: -media(p[1])):
        m = media(notas)
        print(f"{nome:<10}{m:>7.2f} "
              f"{'aprovado' if m >= 7 else 'exame'}")
    medias = [media(n) for n in turma.values()]
    print("-" * 30)
    print(f"{'TURMA':<10}{sum(medias) / len(medias):>7.2f}")

turma = carregar()
turma.update({"Ana": [8.5, 9.0], "Bruno": [6.0, 6.5],
             "Carlos": [7.5, 8.0]})

salvar(turma)
relatorio(carregar())

```

--- SAÍDA ---

ALUNO	MÉDIA	SITUAÇÃO

Ana	8.75	aprovado
Carlos	7.75	aprovado
Bruno	6.25	exame

TURMA	7.58	

+-----

--- #CÓDIGO 16.3 Analisador de arquivo de log -----

```
from collections import Counter

LINHAS = """2026-07-22 10:01 INFO  estação iniciada
2026-07-22 10:03 WARNING sinal fraco em 145.8
2026-07-22 10:04 ERROR falha na porta serial
2026-07-22 10:07 INFO  pacote recebido
2026-07-22 10:09 ERROR falha na porta serial"""

with open("estacao.log", "w", encoding="utf-8") as f:
    f.write(LINHAS)

niveis = Counter()
mensagens_erro = []

with open("estacao.log", encoding="utf-8") as f:
    for linha in f:
        partes = linha.split(maxsplit=3)
        if len(partes) < 4:
            continue
        _, hora, nivel, mensagem = partes
        niveis[nivel] += 1
        if nivel == "ERROR":
            mensagens_erro.append((hora, mensagem.strip()))

print("ocorrências por nível:", dict(niveis))
print("erros registrados:")
for hora, msg in mensagens_erro:
    print(f" {hora} {msg}")
print("mensagem de erro mais comum:",
      Counter(m for _, m in mensagens_erro).most_common(1))
```

--- SAÍDA ---

```
ocorrências por nível: {'INFO': 2, 'WARNING': 1, 'ERROR': 2}
erros registrados:
 10:04 falha na porta serial
 10:09 falha na porta serial
mensagem de erro mais comum: [('falha na porta serial', 2)]
```

--- #CÓDIGO 16.4 Agenda de contatos com menu -----

O mesmo esqueleto (menu, dicionário e persistência em JSON) atende a controle de estoque, cadastro de equipamentos e registro de medidas.

```
import json
from pathlib import Path

ARQUIVO = Path("agenda.json")

def carregar():
    if ARQUIVO.exists():
        return json.loads(ARQUIVO.read_text(encoding="utf-8"))
    return {}

def salvar(agenda):
    ARQUIVO.write_text(json.dumps(agenda, ensure_ascii=False,
                                  indent=2), encoding="utf-8")

def menu():
    agenda = carregar()
    while True:
        print("\n1 incluir | 2 listar | 3 remover | 0 sair")
        opcao = input("opção: ").strip()

        if opcao == "1":
            nome = input("nome: ").strip()
            agenda[nome] = input("telefone: ").strip()
            salvar(agenda)
            print("contato salvo")
        elif opcao == "2":
            if not agenda:
```

```

        print("agenda vazia")
        for nome, fone in sorted(agenda.items()):
            print(f"  {nome:<10} {fone}")
    elif opcao == "3":
        removido = agenda.pop(input("nome: ").strip(), None)
        salvar(agenda)
        print("removido" if removido else "não encontrado")
    elif opcao == "0":
        print("até logo")
        break
    else:
        print("opção inválida")

```

```
menu()
```

```
--- SAÍDA ---
```

```
1 incluir | 2 listar | 3 remover | 0 sair
```

```
opção: 1
```

```
nome: Ana
```

```
telefone: 48 99999-1111
```

```
contato salvo
```

```
1 incluir | 2 listar | 3 remover | 0 sair
```

```
opção: 1
```

```
nome: Bruno
```

```
telefone: 48 98888-2222
```

```
contato salvo
```

```
1 incluir | 2 listar | 3 remover | 0 sair
```

```
opção: 2
```

```
Ana          48 99999-1111
```

```
Bruno        48 98888-2222
```

```
1 incluir | 2 listar | 3 remover | 0 sair
```

```
opção: 3
```

```
nome: Ana
```

```
removido
```

```
1 incluir | 2 listar | 3 remover | 0 sair
```

```
opção: 2
```

```
Bruno        48 98888-2222
```

```
1 incluir | 2 listar | 3 remover | 0 sair
```

```
opção: 0
```

```
até logo
```

>> EXPERIMENTOS PROPOSTOS

- . Acrescente ao jogo da forca um placar salvo em arquivo.
- . Inclua no boletim a opção de lançar notas pelo teclado.
- . Estenda o analisador de log para aceitar o nome do arquivo como argumento de linha de comando (`sys.argv`).
- . Reescreva a agenda usando uma classe `Contato` e uma classe `Agenda`.

=====

|| BLOCO VI

APÊNDICES

Tabelas de consulta rápida durante as aulas e as provas.

=====

=====

[A] REFERÊNCIA DE OPERADORES

=====

Tudo o que aparece entre valores, reunido em uma página.

ARITMÉTICO	EFEITO	EXEMPLO
+ - *	soma, subtração, multiplicação	3 * 4 = 12
/	divisão real	7 / 2 = 3.5
//	divisão inteira	7 // 2 = 3
%	resto	7 % 2 = 1
**	potência	2 ** 10 = 1024

+-- Tabela A.1

ATRIBUIÇÃO	EQUIVALE A
x += 3	x = x + 3
x -= 3	x = x - 3
x *= 3	x = x * 3
x /= 3	x = x / 3
x //= 3	x = x // 3
x %= 3	x = x % 3
x **= 3	x = x ** 3

+-- Tabela A.2

LÓGICO E DE PERTENCIMENTO	RESULTADO
a and b	verdadeiro só quando os dois são verdadeiros
a or b	verdadeiro quando pelo menos um é verdadeiro
not a	inverte o valor
x in colecao	x pertence à coleção
x not in colecao	x não pertence
a is b	a e b são o mesmo objeto
a is not b	objetos diferentes

+-- Tabela A.3

[i] PRECEDÊNCIA, DA MAIOR PARA A MENOR

** > +x -x (unários) > * / // % > + - > comparações > not > and > or. Na dúvida, use parênteses: eles não custam nada e evitam discussão.

[B] MÉTODOS ESSENCIAIS POR TIPO

As chamadas que resolvem a maior parte dos exercícios.

Listas

MÉTODO	FAZ
<code>.append(x)</code>	acrescenta no fim
<code>.insert(i, x)</code>	insere na posição i
<code>.extend(outra)</code>	concatena outra sequência
<code>.remove(x)</code>	remove a primeira ocorrência
<code>.pop(i)</code>	remove e devolve (padrão: o último)
<code>.index(x)</code>	posição da primeira ocorrência
<code>.count(x)</code>	quantas vezes x aparece
<code>.sort()</code>	ordena no lugar
<code>.reverse()</code>	inverte no lugar
<code>.copy()</code>	cópia rasa
<code>.clear()</code>	esvazia

+-- Tabela B.1

Dicionários

MÉTODO	FAZ
<code>.get(k, padrao)</code>	valor sem erro se faltar
<code>.keys()</code>	as chaves
<code>.values()</code>	os valores
<code>.items()</code>	pares chave e valor
<code>.update(outro)</code>	mescla outro dicionário
<code>.pop(k)</code>	remove e devolve o valor
<code>.setdefault(k, v)</code>	devolve o valor, criando se faltar
<code>.copy()</code>	cópia rasa

+-- Tabela B.2

Conjuntos

MÉTODO	FAZ
<code>.add(x)</code>	acrescenta um elemento
<code>.update(iteravel)</code>	acrescenta vários
<code>.remove(x)</code>	remove (erro se faltar)
<code>.discard(x)</code>	remove sem erro
<code>.union(b) ou </code>	união
<code>.intersection(b) ou &</code>	interseção
<code>.difference(b) ou -</code>	diferença
<code>.issubset(b)</code>	está contido em b

+-- Tabela B.3

Funções internas mais usadas

função	FAZ
<code>len(x)</code>	quantidade de elementos
<code>sum(x)</code>	soma
<code>min(x) max(x)</code>	menor e maior
<code>sorted(x, key=, reverse=)</code>	cópia ordenada
<code>enumerate(x, start=)</code>	índice e valor
<code>zip(a, b)</code>	percorre em paralelo
<code>any(x) all(x)</code>	algum ou todos verdadeiros
<code>range(i, f, p)</code>	sequência numérica
<code>type(x) isinstance(x, T)</code>	tipo do objeto
<code>round(x, n)</code>	arredondamento
<code>abs(x)</code>	valor absoluto
<code>repr(x)</code>	representação para depuração

+-+ Tabela B.4

=====

[C] FORMATAÇÃO DE SAÍDA

=====

Referência dos códigos usados dentro de f-strings e de strftime.

ESPECIFICAÇÃO	EFEITO	EXEMPLO	RESULTADO
<code>:.2f</code>	duas casas decimais	<code>f"{3.14159:.2f}"</code>	3.14
<code>:,.2f</code>	separador de milhar	<code>f"{1234.5:,.2f}"</code>	1,234.50
<code>:.1%</code>	percentual	<code>f"{0.256:.1%}"</code>	25.6%
<code>:.2e</code>	notação científica	<code>f"{12345:.2e}"</code>	1.23e+04
<code>:05d</code>	zeros à esquerda	<code>f"{42:05d}"</code>	00042
<code>:<10</code>	alinha à esquerda	<code>f"{'ab':<10} "</code>	ab_____
<code>:>10</code>	alinha à direita	<code>f"{'ab':>10} "</code>	_____ab
<code>:^10</code>	centraliza	<code>f"{'ab':^10} "</code>	____ab____
<code>:^*10</code>	centraliza preenchendo	<code>f"{'ab':^*10}"</code>	****ab****
<code>:x :o :b</code>	hexadecimal, octal, binário	<code>f"{255:x}"</code>	ff
<code>=</code>	mostra a expressão	<code>f"{preco=}"</code>	preco=39.9

+-- Tabela C.1: especificadores de formato (o sublinhado representa espaço).

CÓDIGO	SIGNIFICADO	EXEMPLO
<code>%d</code>	dia com dois dígitos	22
<code>%m</code>	mês com dois dígitos	07
<code>%Y</code>	ano com quatro dígitos	2026
<code>%H:%M:%S</code>	hora, minuto e segundo	14:30:00
<code>%A %B</code>	dia e mês por extenso	depende do idioma do sistema
<code>%j</code>	dia do ano	203

+-- Tabela C.2: códigos de strftime e strptime.

[D] CARTÃO DE CONSULTA RÁPIDA

A sintaxe do manual inteiro em um bloco só, para imprimir e deixar ao lado do teclado.

+-- #CÓDIGO D.1 Resumo da linguagem -----

```
# ---- entrada e saida ----
nome = input("Nome: ")
idade = int(input("Idade: "))
print(f"{nome} tem {idade} anos e média {7.5:.2f}")

# ---- decisao ----
if nota >= 7:      pass
elif nota >= 5:    pass
else:             pass
conceito = "A" if nota >= 9 else "B"

# ---- repeticao ----
for i in range(1, 11): pass
for i, item in enumerate(lista, start=1): pass
for a, b in zip(lista_a, lista_b): pass
while condicao: pass      # break / continue / else

# ---- colecoes ----
lista = [1, 2, 3];          lista.append(4)
tupla = (1, 2, 3)
dicio = {"chave": "valor"};  dicio.get("chave", "padrao")
conj = {1, 2, 3};           conj.add(4)
pares = [x ** 2 for x in lista if x % 2 == 0]

# ---- funcoes ----
def calcular(a, b=1, *args, **kwargs) -> float:
    """Docstring."""
    return a + b

# ---- classes ----
class Aluno:
    def __init__(self, nome):
        self.nome = nome
    def __str__(self):
        return self.nome

# ---- arquivos ----
with open("dados.txt", "w", encoding="utf-8") as f:
    f.write("linha\n")

# ---- execucoes ----
try:      pass
except ValueError as erro: print(erro)
else:     pass
finally:  pass

# ---- ponto de entrada ----
if __name__ == "__main__":
    main()
```

Para continuar estudando

- * Documentação oficial em português: docs.python.org/pt-br/3/
- * Tutorial oficial, capítulos 3 a 9, cobre tudo o que há neste manual com mais profundidade.
- * PEP 8, o guia de estilo: peps.python.org/pep-0008/
- * Exercícios progressivos: Beecrowd, Exercism e Advent of Code.
- * Próximos passos naturais no curso: [numpy](#) e [matplotlib](#) para análise de sinais, [pandas](#) para tabelas, [requests](#) para APIs e [pyserial](#) para instrumentação.

[*] UM MÉTODO DE ESTUDO QUE FUNCIONA

Digite os exemplos em vez de copiar e colar. Depois quebre cada um de propósito: troque um operador, remova uma linha, mude um tipo. Ler a mensagem de erro que aparece ensina mais rápido do que ler o código certo pela terceira vez.

Manual de Experimentos · Linguagem Python · Parte I · Ramon Mayor Martins · IFSC Câmpus São José. Composto em Courier Prime. Todos os exemplos foram executados em Python 3.12 e as saídas reproduzem o resultado real. A Parte II trata dos pacotes científicos (seções 17 a 24) e a Parte III de aprendizado de máquina (seções 25 a 32).